
La Méthode SOBRE

*Architecture d'instruction pour l'usage
optimal des grands modèles de langage*

CONTEXT ENGINEERING

PROMPT ENGINEERING

AGENT SKILLS

INSTRUCTION ARCHITECTURE

D. Dambreville

Version 1.1 · 18 août 2026 · Édition publique

Licence Creative Commons Attribution 4.0 International (CC BY 4.0)

216 références vérifiées

Sommaire

RÉSUMÉ

Ce document propose une méthode complète, opérationnelle et outillée pour concevoir, déployer et maintenir des systèmes fondés sur des grands modèles de langage (LLM). Il repose sur un constat de doctrine devenu explicite en 2026 : la génération courante de modèles ne réclame plus qu'on lui dicte davantage, mais qu'on lui dicte moins et mieux. La méthode s'organise autour de cinq principes — **S**obriété, **O**rthogonalité, **B**ornage, **R**évélation progressive, **É**valuation — et d'une pile d'instruction à sept couches. Elle se conclut par une bibliographie de référence de plus de deux cents entrées vérifiées, classées, annotées et hiérarchisées par ordre de priorité de lecture.

PARTIE I — FONDEMENTS

1	Pourquoi une méthode	4
2	Le contexte comme ressource finie	5
3	Les six modes de défaillance du contexte	6
4	La bascule doctrinale de 2026 : la soustraction	7

PARTIE II — LA MÉTHODE SOBRE

5	Les cinq principes	9
6	La pile d'instruction à sept couches	11
7	Règle d'affectation : où placer une information	12

PARTIE III — COUCHES D'EXÉCUTION

8	Couche L0 — Socle normatif	14
9	Couche L1 — Identité, mission, altitude	14
10	Couche L2 — Contexte de projet persistant	15
11	Couche L3 — Compétences activables (Agent Skills)	17
12	Couche L4 — Interfaces : outils et MCP	19
13	Couche L5 — Récupération juste-à-temps	20
14	Couche L6 — Mémoire de travail et artefacts	21
15	Couche L7 — Évaluation et observabilité	22

PARTIE IV — PROMPT ENGINEERING OPÉRATIONNEL

16	Le prompt canonique en huit blocs	24
17	Table de sélection des techniques	24
18	Ce qui n'est plus rentable en 2026	25

PARTIE V — ORCHESTRATION

19	Choisir entre flux déterministe et agent	27
20	Les six patrons d'orchestration	27

21 Tâches à long horizon 28

PARTIE VI — GOUVERNANCE

22 Sécurité et chaîne d'approvisionnement 29

23 Conformité et traçabilité 30

PARTIE VII — MISE EN ŒUVRE

24 Feuille de route 90 jours 32

25 Catalogue des anti-patrons 34

ANNEXES

Annexe

A Gabarit CLAUDE.md 35

Annexe

B Gabarit SKILL.md 35

Annexe

C Gabarit de prompt 36

Annexe

D Grille d'audit de contexte 36

Annexe

E Glossaire 38

BIBLIOGRAPHIE DE RÉFÉRENCE

Partie I — Fondements

1. Pourquoi une méthode

L'usage des LLM a traversé trois régimes successifs en moins de cinq ans.

Le premier régime, celui de la **formulation**, tenait le résultat pour une fonction du libellé. On y cherchait la tournure heureuse, l'incantation, le mot juste. Ce régime a produit une littérature abondante et une profession éphémère.

Le deuxième régime, celui de l'**accumulation**, a compris que le libellé n'était qu'une fraction de ce que le modèle reçoit. On s'est alors mis à empiler : consignes système de plusieurs milliers de lignes, fichiers de règles, exemples en cascade, garde-fous négatifs. Cette inflation répondait à une réalité — les modèles de 2023-2024 avaient besoin qu'on leur interdise explicitement ce qu'ils faisaient mal — mais elle a engendré une dette d'instruction considérable.

Le troisième régime, celui de l'**architecture**, est le nôtre. Il postule que la performance dépend moins de ce qu'on écrit que de la manière dont on **organise, hiérarchise, borne et retire** l'information disponible. Il traite le contexte comme un artefact d'ingénierie : versionné, testé, budgété, élagué.

Le tournant public de ce troisième régime est daté. Le 24 juillet 2026, Anthropic publiait *The new rules of context engineering for Claude 5 generation models*, sous la plume de Thariq Shihpar. L'entreprise y expose avoir retiré plus de 80 % de la consigne système de Claude Code pour les modèles de génération 5 — Opus 5, Fable 5, Sonnet 5 — sans perte mesurable sur ses évaluations de code. Un éditeur de modèles expliquant comment il a supprimé la majeure partie de son propre travail d'ingénierie de prompt : l'événement est suffisamment inhabituel pour valoir doctrine.

Cette méthode formalise ce troisième régime. Elle n'est ni une liste de trucs ni un catalogue de techniques : elle propose une **architecture**, c'est-à-dire un ensemble de couches aux responsabilités disjointes, une règle d'affectation qui dit où placer chaque information, et un dispositif de preuve qui interdit d'ajouter sans mesurer.

1.1 Périmètre

La méthode s'applique :

- aux **assistants conversationnels** configurés par consigne système ou par projet ;
- aux **agents de code** pilotés par fichiers de contexte (CLAUDE.md, AGENTS.md) ;
- aux **agents métier** construits sur API, outils et connecteurs ;
- aux **chaînes de traitement** documentaires et analytiques.

Elle est écrite en référence à l'écosystème Claude, dont la documentation est la plus explicite sur ces sujets, mais ses principes sont transposables : le format Agent Skills est devenu un standard ouvert en décembre 2025, adopté depuis par une quarantaine de clients agentiques, et les mécanismes de contexte sont structurellement analogues chez les autres éditeurs.

1.2 Ce que la méthode ne fait pas

Elle ne traite pas de l'entraînement, de l'ajustement fin, ni de l'hébergement de modèles. Elle ne remplace pas une évaluation métier : elle en fournit le cadre. Elle ne dispense d'aucune obligation réglementaire.

2. Le contexte comme ressource finie

2.1 Définition

On appelle **contexte** l'ensemble des jetons présents à l'entrée du modèle au moment de l'échantillonnage. Il comprend, dans l'ordre d'assemblage habituel :

Composant	Origine	Persistance
Consigne système	éditeur ou intégrateur	permanente
Politiques et garde-fous	organisation	permanente
Métadonnées de compétences	fichiers SKILL.md	permanente (métadonnées seules)
Définitions d'outils	serveurs MCP, API	permanente
Fichiers de contexte projet	CLAUDE.md, AGENTS.md	permanente (par portée)
Mémoire persistante	fichiers, base	permanente ou sélective
Historique de conversation	session	croissante
Résultats d'outils	exécution	croissante
Documents récupérés	RAG, lecture fichier	ponctuelle
Message de l'utilisateur	interaction	ponctuelle

Le prompt de l'utilisateur, objet de tant d'attention, en constitue souvent moins de cinq pour cent.

2.2 Le budget d'attention

Deux propriétés gouvernent l'économie du contexte.

La finitude. La fenêtre a une borne dure. Même portée à un million de jetons, elle reste une enveloppe, et chaque jeton consommé l'est au détriment d'un autre. Tout ajout comporte un coût d'opportunité.

La dégradation graduelle. Bien avant la borne dure, la capacité du modèle à exploiter fidèlement ce qui lui est fourni décroît à mesure que l'entrée s'allonge. Ce phénomène, documenté sous le nom de *context rot* par le rapport technique de Chroma (Hong, Troynikov & Huber, 2025), prolonge une observation antérieure : les informations situées au milieu d'un long contexte sont moins bien exploitées que celles placées aux extrémités (Liu *et al.*, *Lost in the Middle*, TACL 2024). Les évaluations de type *needle in a haystack* et les batteries RULER ou LongBench ont établi que la « taille de contexte utile » d'un modèle est systématiquement inférieure à sa taille nominale.

La conséquence pratique est brutale et contre-intuitive : **un contexte plus long peut produire un résultat plus mauvais**, à modèle constant. La compression n'est pas une concession budgétaire, c'est un levier de qualité.

2.3 L'objectif formulé correctement

La question à poser n'est pas « qu'est-ce que je peux ajouter pour améliorer le résultat ? », mais :

Quelle est la configuration minimale de jetons à signal élevé qui maximise la probabilité du comportement attendu ?

Le mot « minimale » ne signifie pas « courte ». Il signifie qu'aucun jeton présent ne peut être retiré sans perte mesurable. C'est une définition testable, et c'est tout l'intérêt.

3. Les six modes de défaillance du contexte

Un diagnostic précis précède toute correction. La littérature praticienne a stabilisé un vocabulaire des pathologies contextuelles ; on en retient six.

3.1 Empoisonnement (*poisoning*)

Une erreur, une hypothèse fausse ou une hallucination produite par le modèle demeure dans l'historique et acquiert le statut de fait établi. Les tours suivants raisonnent dessus. Le mécanisme est auto-renforçant : plus la session s'allonge, plus l'erreur est citée, plus elle paraît fondée.

Signature : le modèle défend une affirmation qu'il a lui-même inventée trente tours plus tôt. *Remède* : purge sélective, redémarrage sur contexte propre avec état reconstruit depuis une source de vérité externe, jamais depuis le résumé de la session.

3.2 Distraction

Le volume d'information disponible dilue le signal. Les résultats d'outils bruts, les journaux d'erreur résolus, les fichiers lus « au cas où » occupent l'attention sans porter d'information utile.

Signature : le modèle produit un travail correct mais hors sujet, ou traite une sous-partie en ignorant la demande principale. *Remède* : pagination et troncature des retours d'outils, purge des résultats obsolètes, budget de jetons par appel.

3.3 Confusion

Trop d'outils, trop de compétences, trop de chemins possibles. Le modèle choisit mal parce que l'espace de choix est mal cloisonné. La règle empirique est nette : si un ingénieur humain ne peut pas dire avec certitude quel outil employer dans une situation donnée, le modèle ne fera pas mieux.

Signature : appels d'outils inappropriés, oscillation entre deux outils quasi identiques. *Remède* : élagage du jeu d'outils, désambiguïsation des descriptions, fusion des doublons, recherche d'outils différée.

3.4 Conflit (*clash*)

Deux instructions issues de deux couches se contredisent. Le cas typique : une consigne système hérite de garde-fous conçus pour une génération antérieure de modèles, tandis qu'un fichier de projet énonce la règle inverse. C'est le mode de défaillance que la doctrine 2026 identifie comme le plus coûteux, précisément parce qu'il est invisible : rien ne signale une contradiction, le modèle arbitre silencieusement et son arbitrage varie.

Signature : comportement instable d'une session à l'autre sur une même demande. *Remède* : audit d'orthogonalité — voir §7 et l'annexe D.

3.5 Corrosion (*context rot*)

Dégradation progressive de la fidélité de rappel avec l'allongement de l'entrée. Elle n'est pas une erreur de conception mais une propriété du substrat ; elle se gère, elle ne se corrige pas.

Signature : une information présente dans le contexte est ignorée ou déformée, alors qu'elle serait correctement traitée dans un contexte court. *Remède* : compaction, hiérarchisation par position, externalisation de la mémoire, découpage en sous-agents à contexte isolé.

3.6 Sur-contrainte (*overconstraint*)

Pathologie propre à la génération courante et longtemps invisible : l'accumulation de contraintes négatives, d'exemples restrictifs et de règles impératives réduit l'espace d'exploration du modèle sous le seuil où son jugement est meilleur que notre prescription.

Signature : le modèle produit un travail littéralement conforme et manifestement inadéquat. Il obéit contre l'intention. *Remède* : soustraction (§4), reformulation en intention plutôt qu'en interdiction.

4. La bascule doctrinale de 2026 : la soustraction

Six bascules ont été formalisées par Anthropic en juillet 2026. Les voici, reformulées et commentées.

4.1 Des règles au jugement

Avant : énumérer les interdictions. « N'écris jamais de commentaire. Ne crée jamais de fichier de planification sans demande explicite. »

Après : énoncer l'intention et laisser le modèle inférer. « Écris du code qui se lit comme le code environnant : même densité de commentaires, même nomenclature, mêmes idiomes. »

La règle négative était nécessaire quand le modèle produisait effectivement des commentaires inutiles. Elle devient nuisible quand il sait juger, car elle lui interdit le bon comportement dans les cas où le commentaire est justifié.

Exception impérative. La bascule ne s'applique **pas** aux domaines où la conséquence d'une erreur est asymétrique et grave : conformité réglementaire, secret professionnel, plafonds financiers, autorité de signature, sécurité des personnes. Là, la règle explicite reste indispensable. La doctrine officielle emploie la formule « sauf dans les domaines de haute importance » ; il convient de la lire comme une clause de sauvegarde, non comme une concession.

4.2 Des exemples à la conception d'interface

Avant : multiplier les exemples pour cadrer la sortie.

Après : concevoir des interfaces expressives — paramètres d'outils sans ambiguïté, énumérations fermées, schémas stricts, noms explicites (`user_id` et non `user`).

Un exemple enseigne un cas ; une interface bien typée interdit une classe d'erreurs. L'exemple restreint aussi l'espace d'exploration : le modèle tend à reproduire la forme fournie plutôt qu'à chercher la meilleure. On conserve donc un petit ensemble d'exemples **canoniques et diversifiés**, jamais une liste de cas particuliers.

4.3 De l'exhaustivité à la révélation progressive

Avant : tout charger dans la consigne système.

Après : trois paliers de chargement — découverte, activation, exécution. La consigne système ne porte que ce qui vaut pour toutes les requêtes ; le reste attend sur disque et n'entre qu'appelé.

4.4 Du prompt épais à l'artefact épais

Formule mnémotechnique due à l'équipe Claude Code : **prompts minces, artefacts et contexte épais, compétences minces**.

Le prompt exprime une intention ponctuelle et doit rester léger. Ce qui doit être épais, c'est le **matériau** : la spécification, le corpus de référence, le jeu de données, le document de conception. Une compétence, elle, doit rester mince : c'est une procédure, pas une encyclopédie.

4.5 De la duplication à l'orthogonalité

Chaque information réside en **un seul lieu** de la pile. La duplication n'est pas une redondance de sécurité : c'est une source de conflit différé, car les deux copies divergeront.

4.6 De l'accumulation à l'audit périodique

Le contexte est du code. Il se relit, se refactorise, se supprime. Il vieillit particulièrement mal, car il a été écrit pour un modèle qui n'existe plus. Tout changement de génération de modèle doit déclencher un audit de contexte — pas une migration d'identifiant de modèle.

Partie II — La méthode SOBRE

5. Les cinq principes

S — Sobriété

Chaque jeton doit gagner sa place.

Énoncé. Aucun élément n'entre dans le contexte sans qu'on puisse nommer le comportement qu'il produit et démontrer, par retrait, qu'il le produit effectivement.

Test opérationnel — le test de retrait. Pour chaque bloc du contexte, retirez-le et faites tourner la suite d'évaluation. Si le score ne bouge pas, le bloc est mort : il coûte des jetons, de la latence, de l'argent, et il augmente la surface de conflit. Supprimez-le.

Ce qui échoue systématiquement au test de retrait : - ce que le modèle peut déduire de l'environnement (« ce projet est en TypeScript » — il voit le `tsconfig.json`) ; - ce qui figure déjà dans un fichier de configuration lisible (« lance les tests avec `npm test` » — c'est dans le `package.json`) ; - les préférences stylistiques mineures ; - les garde-fous hérités d'une génération antérieure ; - les politesses, menaces, primes fictives et autres leviers émotionnels.

Ce qui y survient toujours : - ce que le modèle ne peut structurellement pas inférer : décisions arbitraires de l'organisation, conventions internes non documentées, chausse-trapes du dépôt, historique des choix ; - les invariants normatifs et réglementaires ; - la voix de marque, les planchers tarifaires, les frontières de confidentialité, l'autorité de validation.

O — Orthogonalité

Une information, un lieu.

Énoncé. Les couches de la pile ont des responsabilités disjointes. Aucune information n'apparaît dans deux couches. Aucune couche n'énonce une règle que sa couche supérieure contredit.

Test opérationnel — la matrice de conflit. Construisez la liste de toutes les assertions normatives de votre pile (une ligne par instruction impérative, avec sa provenance). Recherchez : doublons exacts, quasi-doublons de formulation divergente, contradictions directes, contradictions conditionnelles. Chaque occurrence est un défaut à corriger, pas une observation à noter.

Corollaire de portée. Quand une instruction ne vaut que pour un sous-ensemble du travail, elle descend dans la hiérarchie : un fichier de contexte par répertoire vaut mieux qu'un fichier racine encyclopédique. Le principe est celui des fichiers de configuration hiérarchiques : la portée la plus fine qui suffit.

B — Bornage (l'altitude juste)

Ni pilote automatique, ni carte blanche.

Énoncé. Chaque instruction doit être formulée à la bonne altitude : suffisamment spécifique pour orienter, suffisamment souple pour laisser au modèle son jugement.

Deux échecs symétriques bornent la zone utile :

- **trop bas** : logique conditionnelle codée en dur, arbres de décision explicites, énumération de cas limites. Résultat : fragilité, coût de maintenance, contradictions internes ;
- **trop haut** : généralités inopérantes, présupposition d'un contexte partagé qui n'existe pas. Résultat : dérive, absence de signal.

Test opérationnel. Faites lire votre instruction à un collaborateur compétent mais nouveau dans le projet. S'il vous demande « mais dans tel cas, je fais quoi ? », vous êtes trop haut. S'il vous dit « pourquoi m'interdire ça, c'est évidemment utile ici », vous êtes trop bas.

Rédaction. Employez l'impératif, non le descriptif. « Valide le schéma avant l'insertion » vaut mieux que « le schéma devrait normalement être validé ». Adoptez le vocabulaire normatif du RFC 2119 — **DOIT**, **NE DOIT PAS**, **DEVRAIT**, **PEUT** — et respectez-en la sémantique : réservez **DOIT** aux invariants réels, sous peine d'inflation impérative.

R — Révélation progressive

Ce qui n'est pas nécessaire maintenant reste sur disque.

Énoncé. L'information est organisée en paliers de chargement, et ne monte d'un palier que lorsque la tâche l'exige.

Le mécanisme canonique est celui des Agent Skills, à trois paliers :

Palier	Contenu chargé	Moment	Coût observé
Découverte	<code>name</code> + <code>description</code> du frontmatter YAML	démarrage, pour toutes les compétences	ordre de 50 à 250 jetons par compétence, médiane voisine de 80
Activation	corps du fichier <code>SKILL.md</code>	quand la description correspond à la tâche	recommandé sous 5 000 jetons
Exécution	fichiers de <code>references/</code> , <code>assets/</code> , exécution de <code>scripts/</code>	à la demande, pendant le travail	variable, nul si non appelé

La propriété remarquable est l'**élasticité** : on peut installer cent compétences sans les payer, tant qu'elles ne se déclenchent pas. La révélation progressive transforme un coût fixe en coût marginal.

Elle s'applique bien au-delà des compétences : références légères (chemins de fichiers, requêtes enregistrées, identifiants) plutôt que contenus inlinés ; tables des matières plutôt que documents ; recherche d'outils différée plutôt que catalogue exhaustif.

Note d'histoire des idées. Le terme est emprunté à l'ergonomie logicielle, où il désigne depuis les années 1990 la stratégie d'exposition graduelle des fonctionnalités d'une interface. Le transfert au domaine cognitif des agents est exact : dans les deux cas, il s'agit de protéger une ressource attentionnelle limitée.

É — Évaluation

Rien n'entre dans la pile sans preuve. Rien n'y reste sans preuve renouvelée.

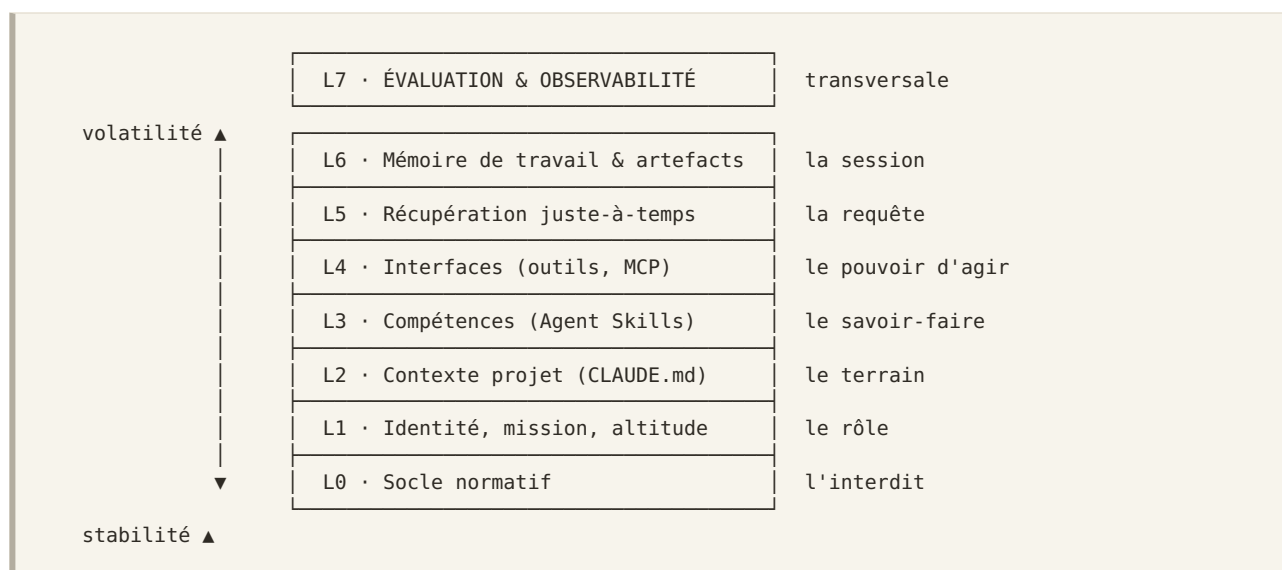
Énoncé. Toute modification du contexte est accompagnée d'une évaluation qui en démontre l'effet, et versée à une suite de non-régression.

C'est le principe qui rend les quatre autres exécutoires. Sans évaluation, la sobriété devient une opinion, l'orthogonalité un vœu, le bornage un débat de goût. Avec évaluation, ce sont des mesures.

Le renversement méthodologique. On n'écrit pas le contexte puis l'évaluation : **on écrit l'évaluation d'abord**. Le jeu de cas définit ce que « fonctionner » signifie ; le contexte est la variable qu'on ajuste pour y satisfaire. C'est la transposition du développement piloté par les tests, et elle est plus justifiée encore ici, puisque le comportement du système n'est pas déductible de sa spécification.

6. La pile d'instruction à sept couches

L'*Instruction Architecture* organise le contexte en sept couches de responsabilité disjointe, de la plus stable à la plus volatile, plus une couche transversale.



Couche	Question à laquelle elle répond	Durée de vie	Élaguable
L0 Socle normatif	Qu'est-il interdit, en toute circonstance ?	années	jamais
L1 Identité & mission	Qui suis-je, pour quoi faire, à quelle altitude ?	trimestres	avec précaution
L2 Contexte projet	Sur quel terrain travaille-t-on ?	semaines	oui, agressivement
L3 Compétences	Comment exécute-t-on cette procédure ?	mois	par palier
L4 Interfaces	Que peut-on faire, et selon quel contrat ?	mois	par élagage du jeu d'outils
L5 Récupération	De quoi ai-je besoin, maintenant, pour cette requête ?	la requête	par construction
L6 Mémoire de travail	Où en suis-je ?	la session	par compaction
L7 Évaluation	Est-ce que cela fonctionne, et le sait-on ?	permanente	jamais

Invariant d'architecture : la volatilité croît de bas en haut, et la spécificité avec elle. Une information hautement spécifique placée dans une couche basse est un défaut de conception, car elle sera payée à chaque requête sans servir à presque aucune.

7. Règle d'affectation : où placer une information

C'est la question la plus fréquente et la plus mal résolue en pratique. Voici l'arbre de décision.

Une information nouvelle se présente.

1. Le modèle peut-il l'inférer de l'environnement ?
 - └ OUI → NE PAS L'ÉCRIRE. Fin.
 - └ NON → continuer
2. Est-elle normative et sa violation grave ?
 - └ OUI → L0 (socle normatif). Formulation impérative explicite. Fin.
 - └ NON → continuer
3. Vaut-elle pour toutes les requêtes de tous les projets ?
 - └ OUI → L1 (identité & mission). Fin.
 - └ NON → continuer
4. Vaut-elle pour toutes les requêtes d'un projet donné ?
 - └ OUI → L2 (contexte projet), à la portée la plus fine possible
(fichier de répertoire plutôt que fichier racine). Fin.
 - └ NON → continuer
5. Est-ce une procédure, réutilisable, déclenchable par description ?
 - └ OUI → L3 (compétence). SKILL.md mince + références en palier 3. Fin.
 - └ NON → continuer
6. Est-ce un pouvoir d'agir sur un système externe ?
 - └ OUI → L4 (outil / serveur MCP). Encoder dans le schéma,
pas dans la prose. Fin.
 - └ NON → continuer
7. Est-ce un contenu volumineux, consultable à la demande ?
 - └ OUI → L5 (récupération JIT). Exposer une référence légère,
jamais le contenu. Fin.
 - └ NON → continuer
8. Est-ce un état transitoire de la tâche en cours ?

- └ OUI → L6 (mémoire de travail). Fichier externe, pas historique. Fin.
- └ NON → Vous ne savez pas ce que c'est. Ne l'écrivez pas.

Le pas 1 est le plus important et le plus négligé. La majorité de la dette d'instruction observée en production consiste à expliquer au modèle ce qu'il lit déjà.

Le pas 8 est un garde-fou. Une information qui ne trouve pas sa couche est le symptôme d'un besoin mal formulé. La bonne réponse est de reformuler le besoin, non d'ajouter la ligne quelque part.

Partie III — Couches d'exécution

8. Couche L0 — Socle normatif

Contenu

Les invariants dont la violation est inacceptable et dont le modèle ne peut pas déduire l'existence :

- obligations légales et réglementaires du domaine (secteur régulé, données personnelles, communication financière) ;
- frontières de confidentialité : ce qui ne sort jamais du périmètre, vers qui, sous quelle forme ;
- autorité de validation : ce qui exige une signature humaine avant effet ;
- plafonds et planchers : montants, remises, engagements ;
- interdits absolus propres à l'organisation.

Règles de rédaction

1. **Impératif explicite, pas d'intention.** C'est l'exception assumée au principe de bascule vers le jugement (§4.1). On n'écrit pas « fais preuve de discernement sur les données clients » mais « NE DOIT PAS inclure de donnée nominative de client dans une sortie destinée à un tiers ».
2. **Une règle par ligne, une conséquence par règle.** L'énumération facilite l'audit.
3. **Justification brève et attachée.** Une règle sans motif sera supprimée au premier élagage ; une règle avec motif sera conservée à bon escient. Format : `RÈGLE – motif – source normative`.
4. **Traçabilité.** Chaque règle porte une référence : article de règlement, décision interne datée, politique numérotée.
5. **Aucune exception implicite.** Si une exception existe, elle est écrite. Une règle absolue qu'on contourne en pratique enseigne au modèle que les règles absolues sont négociables.

Ce que L0 ne contient pas

Ni préférence de style, ni convention technique, ni garde-fou de performance. Le socle normatif n'est pas la poubelle des contraintes ; c'est le petit noyau incompressible. Un L0 de plus de trente lignes est probablement pollué.

9. Couche L1 — Identité, mission, altitude

Contenu

Ce qui vaut pour toute requête, tous projets confondus : rôle, objectif, critères de réussite, registre de communication, format de sortie par défaut, comportement en cas d'incertitude.

Structure recommandée

Segmentez explicitement, par balises XML ou titres Markdown. La délimitation aide le modèle à situer chaque information — et vous aide à auditer.

```

<mission>
  Ce que le système est là pour accomplir, en trois phrases.
</mission>

<criteres_de_reussite>
  Comment on reconnaît un bon résultat. Ce que l'on préfère
  quand deux qualités s'opposent.
</criteres_de_reussite>

<registre>
  Voix, niveau de langue, longueur par défaut, langue de réponse.
</registre>

<conduite_en_incertitude>
  Que faire quand l'information manque : demander, supposer et
  signaler, ou refuser.
</conduite_en_incertitude>

<format_de_sortie>
  Structure par défaut. Ce qui déclenche un autre format.
</format_de_sortie>

```

La question de l'altitude

C'est ici qu'elle se jogue le plus visiblement. Un exemple appliqué au domaine de la veille financière :

- **Trop bas** : « Si l'indicateur est un taux directeur, mentionne la banque centrale émettrice, la date de décision, le consensus de marché antérieur, l'écart au consensus, et conclus par une phrase sur les implications obligataires. »
- **Trop haut** : « Sois pertinent et rigoureux. »
- **Juste** : « Chaque affirmation quantitative porte sa source et sa date. Distingue explicitement le fait constaté, le consensus de marché et ton analyse. Quand un chiffre est provisoire, dis-le. »

L'altitude juste énonce des **invariants de qualité**, non des gabarits de rédaction.

Le paragraphe le plus rentable

Formulez explicitement **la préférence en cas d'arbitrage**. Presque tous les systèmes échouent non par ignorance mais par mauvais arbitrage entre deux qualités désirables : exhaustivité contre concision, prudence contre utilité, vitesse contre vérification. Une phrase qui ordonne ces préférences vaut cinquante lignes de règles particulières.

10. Couche L2 — Contexte de projet persistant

Fichiers `CLAUDE.md`, `AGENTS.md`, documents de contexte de projet.

Le critère unique

N'écrivez que ce que le modèle ne peut pas découvrir seul.

Ce fichier n'est pas une documentation de projet. La documentation s'adresse à des humains qui ne peuvent pas lire tout le dépôt ; le modèle, lui, peut. Il n'est pas non plus un tutoriel. C'est une **note de passation de chausse-trapes**.

Contenu utile, par ordre de rendement décroissant

1. **Les pièges non déductibles.** Les décisions surprenantes et leur motif. « Le module de facturation duplique la logique de taxes du module de commande — c'est délibéré, la refonte est planifiée, ne les factorisez pas. » Voilà la ligne qui sauve une heure et qui ne s'infère de rien.
2. **Les conventions arbitraires.** Ce qui est un choix, non une déduction : nomenclature interne, découpage des branches, format des messages de validation quand il est inhabituel.
3. **L'intention du projet, en trois lignes.** À quoi sert ce dépôt, pour qui, dans quelle phase.
4. **Les zones interdites.** Ce qui ne doit pas être modifié, et pourquoi. Code généré, migrations passées, fichiers sous contrainte externe.
5. **Le vocabulaire maison.** Acronymes, sigles, noms de code, surnoms d'équipe. Décodage indispensable : un modèle qui ne sait pas ce que désigne un sigle interne ne peut pas comprendre la demande qui l'emploie.
6. **Les points d'entrée non évidents.** Là où commencer quand on cherche telle chose, si l'arborescence ne le dit pas.

Contenu à ne jamais écrire

À supprimer	Motif
« Ce projet utilise TypeScript / Python / React »	visible dans les fichiers de configuration
« Lance les tests avec <code>npm test</code> »	dans le <code>package.json</code>
« Le code se trouve dans <code>src/</code> »	l'arborescence le dit
« Utilise 2 espaces d'indentation »	dans le fichier de style, et déductible du code
« Écris du code propre et lisible »	inopérant
« Ne casse pas les tests existants »	présupposé
Une description de chaque répertoire	le modèle lit l'arborescence
Un journal des modifications	il y a un historique de versions pour ça

Volumétrie

Les guides convergents de 2026 recommandent **moins de 150 à 200 lignes** pour un fichier racine. Un fichier de 500 lignes n'est pas un fichier riche : c'est un fichier non audité. Un fichier de 4 000 lignes est une dette qui se paie à chaque requête et livre un contexte pollué.

Hiérarchie par portée

Préférez plusieurs fichiers courts, situés au plus près de leur objet, à un fichier racine encyclopédique.

```

CLAUDE.md                # intention du projet, pièges globaux (< 100 l.)
├─ services/paiement/CLAUDE.md # spécificités du domaine paiement (< 50 l.)
├─ services/ingestion/CLAUDE.md # contraintes de la chaîne d'ingestion (< 50 l.)
└─ infra/CLAUDE.md          # règles de déploiement (< 40 l.)

```

Le fichier de répertoire n'est chargé que lorsque le travail y touche. C'est de la révélation progressive appliquée au contexte de projet — et c'est le gain le plus facile à obtenir dans un dépôt existant.

Rituel de maintenance

- **Déclencheur obligatoire** : tout changement de génération de modèle. Le fichier a été écrit pour un modèle qui n'existe plus.
- **Cadence** : relecture mensuelle, avec test de retrait ligne à ligne sur la suite d'évaluation.
- **Outillage** : dans l'écosystème Claude Code, la commande `/doctor` (également `claude doctor` en ligne de commande) audite la pile de contexte et signale les sections qu'elle suspecte redondantes. Elle propose ; elle ne décide pas. Lisez chaque suggestion, appliquez-la ou annotez la raison de la conserver. Les règles de conformité restent ; « merci d'utiliser une indentation de deux espaces » probablement pas.

11. Couche L3 — Compétences activables (Agent Skills)

11.1 Nature

Une compétence est un **dossier** contenant un fichier `SKILL.md`, chargé à la demande, qui transmet à l'agent une procédure spécialisée. Le format, créé par Anthropic (annonce en octobre 2025), a été publié comme standard ouvert le 18 décembre 2025 et est depuis adopté par une quarantaine de clients agentiques.

```
ma-competence/
├─ SKILL.md          # OBLIGATOIRE – frontmatter YAML + instructions
├─ scripts/          # optionnel – code exécutable (Python, Bash, JS)
├─ references/       # optionnel – documentation chargée à la demande
├─ assets/           # optionnel – gabarits, schémas, données
└─ evals/           # recommandé – jeu de cas de validation
```

Le frontmatter n'exige que deux champs : `name` et `description`.

11.2 La description : le point de levier

C'est la partie la plus rentable et la plus bâclée de tout l'exercice. La description est le **seul** élément, avec le nom, qui reste en permanence dans le contexte ; c'est aussi contre elle que l'agent apparie la demande pour décider s'il active la compétence. Une description médiocre produit deux pathologies : la compétence ne se déclenche jamais, ou elle se déclenche toujours.

Règle de composition. La description doit énoncer **ce que la compétence fait et quand l'employer**. Le second membre est le plus souvent omis, et c'est lui qui gouverne le déclenchement.

```
# Insuffisant – pas de condition de déclenchement
description: Traite les fichiers PDF.

# Insuffisant – condition trop large, déclenchement parasite
description: Utiliser pour tout travail sur des documents.

# Correct
description: >-
  Extrait le texte et les tableaux de fichiers PDF, remplit des
  formulaires, fusionne des documents. Utiliser lorsque l'utilisateur
  mentionne un PDF, un formulaire à remplir, ou une extraction
  documentaire.
```

Vérification. Rédigez dix formulations plausibles de demande : cinq qui doivent déclencher la compétence, cinq qui ne doivent pas. Testez. Le taux de déclenchement correct est une métrique, pas une impression.

11.3 Rédaction du corps

Règle	Justification
Rester sous ~500 lignes / 5 000 jetons	au-delà, la compétence pollue le contexte qu'elle est censée économiser
Impératif, jamais descriptif	« Valide le schéma » et non « le schéma est habituellement validé »
Étapes séquentielles numérotées	réduit les omissions dans les procédures longues
Liste de contrôle recopiable	l'agent la reproduit et la coche, ce qui matérialise l'avancement
Boucles de vérification explicites	Exécuter → Vérifier → Corriger → Recommencer
Références à un seul niveau de profondeur	garantit que l'agent lit des fichiers complets, sans chaînage
Table des matières pour toute référence > 100 lignes	l'agent voit la portée disponible même en lecture partielle
Une compétence, une responsabilité	la compétence fourre-tout ne se déclenche jamais correctement

Externalisez le calcul. Ce qu'un script fait de façon déterministe ne doit pas être décrit en prose pour que le modèle le refasse. Un script dans `scripts/` s'exécute sans consommer de contexte pour son propre code : c'est le meilleur rapport fiabilité/jetons de toute l'architecture. Règle : **formules et code plutôt que valeurs calculées.**

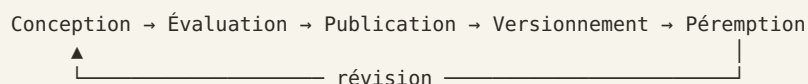
11.4 Compétences, outils, MCP : la répartition

Confusion fréquente, résolue par une formule simple :

Mécanisme	Répond à	Nature
Compétence	<i>comment faire</i>	savoir procédural, texte + scripts
Outil / MCP	<i>avec quoi agir</i>	pouvoir d'action, contrat d'appel
Sous-agent	<i>par qui le faire</i>	isolation de contexte
Contexte projet	<i>où on est</i>	connaissance du terrain

Une compétence dit à l'agent comment mener un audit de conformité ; un serveur MCP lui donne l'accès à la base documentaire. Les deux sont complémentaires et ne se substituent pas. Écrire l'une ne dispense pas de l'autre.

11.5 Cycle de vie



1. **Conception.** Une compétence naît d'une répétition constatée : si vous avez retapé la même consigne trois fois, elle est mûre. Pas avant : une compétence prématurée est une contrainte gratuite.
2. **Évaluation.** Jeu de cas dans `evals/`, incluant les cas de non-déclenchement. Aucune publication sans jeu de cas.
3. **Publication.** Dépôt versionné, distribution par greffon ou par réglages gérés à l'échelle de l'organisation.

4. **Versionnement.** Versionnage sémantique, journal des modifications tenu. Une compétence est une dépendance : elle a des consommateurs qui doivent savoir ce qui a changé.
5. **Péremption.** Date de revue obligatoire. Une compétence non révisée depuis deux générations de modèles est présumée sur-contrainte. Le réflexe correct est de la **raccourcir**, non de l'enrichir.

11.6 Sécurité

Une compétence est du code exécutable accompagné d'instructions. Une compétence malveillante peut exfiltrer des données, invoquer des outils à contre-emploi, ou détourner l'agent de son objet déclaré.

- **N'installez que depuis des sources de confiance** : les vôtres, ou celles de l'éditeur.
- **Auditez l'intégralité du dossier** : `SKILL.md`, scripts, références, ressources. Cherchez les appels réseau inattendus, les lectures de fichiers hors périmètre, les écritures, les dépendances externes non épinglées.
- **Traitez le contenu de `references/` comme des données, non comme des instructions.** Une compétence légitime dont les fichiers de référence sont modifiables par un tiers devient un vecteur d'injection indirecte.
- **Épinglez les dépendances** et interdisez les URL dynamiques.
- Le champ `allowed-tools` restreint l'accès aux outils ; employez-le systématiquement en principe de moindre privilège.

12. Couche L4 — Interfaces : outils et MCP

12.1 Le glissement de charge

La bascule §4.2 déplace l'effort de la prose vers le schéma. Une contrainte encodée dans une interface est vérifiée par construction ; la même contrainte écrite en prose est une suggestion probabiliste. Préférez toujours le schéma.

12.2 Règles de conception

1. **Élaguer.** Le critère est celui du §3.3 : si un ingénieur humain ne peut pas trancher entre deux outils, retirez-en un ou fusionnez-les. Un jeu de dix outils bien découpés surpasse un jeu de quarante outils qui se recouvrent.
2. **Nommer sans ambiguïté.** `user_id`, non `user`. `created_after_iso8601`, non `date`. Le nom du paramètre est une instruction gratuite : elle ne coûte presque rien et elle est toujours lue.
3. **Fermer les énumérations.** Toute valeur appartenant à un ensemble fini est déclarée comme énumération. C'est la manière la moins coûteuse d'éliminer une classe entière d'erreurs.
4. **Documenter la description comme une consigne.** La description d'un outil entre dans le contexte : elle mérite le même soin qu'une consigne système. Écrivez-la comme vous l'écririez pour un nouvel arrivant compétent : rendez explicite le contexte implicite — formats de requête particuliers, terminologie interne, relations entre ressources.
5. **Borner les retours.** Pagination, sélection d'intervalle, filtrage, troncature, avec des valeurs par défaut raisonnables. Claude Code, par exemple, plafonne par défaut les réponses d'outils à 25 000 jetons. Un outil qui peut renvoyer un million de jetons est un vecteur de distraction.
6. **Rendre les erreurs actionnables.** Un message d'erreur est une consigne adressée au modèle. « Paramètre `start_date` invalide : format attendu `YYYY-MM-DD`, reçu `12/03/2026` » lui permet de corriger. Un code d'erreur opaque ou une trace d'exécution ne lui permet rien.
7. **Orienter vers l'efficacité.** Vous pouvez explicitement encourager les stratégies économes : plusieurs recherches ciblées plutôt qu'une requête large, appel programmatique plutôt que séquence d'appels unitaires.

8. **Différer le catalogue.** Quand le nombre d'outils devient grand, recourez à une recherche d'outils : les définitions ne sont chargées qu'appelées. Même logique de révélation progressive qu'en L3.

12.3 Appel programmatique

Quand une tâche enchaîne de nombreux appels dont seuls les résultats finaux importent, faites orchestrer l'agent **par du code** plutôt que par une séquence d'appels individuels : le code consomme les sorties intermédiaires et ne renvoie que le résultat traité. Le gain contextuel est d'un ordre de grandeur sur les traitements en lot.

13. Couche L5 — Récupération juste-à-temps

13.1 Deux stratégies, un continuum

	Pré-chargement	Juste-à-temps (JIT)
Principe	tout le pertinent est chargé avant l'inférence	l'agent découvre et charge en cours de route
Force	latence prévisible, contrôle complet	contexte minimal, exploration adaptative
Faiblesse	dilution, coût fixe élevé	latence variable, risque d'errance
Convient à	corpus stable, périmètre connu	dépôts, arborescences, bases documentaires vastes

Le patron recommandé est hybride : pré-chargez le petit noyau incontournable ; exposez tout le reste par **références légères** — chemins de fichiers, requêtes enregistrées, identifiants, tables des matières — que l'agent résout à la demande.

Un chemin de fichier coûte quinze jetons. Le fichier en coûte cinq mille. Si la probabilité d'en avoir besoin est de vingt pour cent, l'arithmétique est sans appel.

13.2 Hygiène de récupération

- **Chunking sémantique**, non par longueur fixe. Un fragment coupé au milieu d'un raisonnement est inexploitable.
- **Recherche hybride** — lexicale (BM25) et dense — puis **réordonnement**. La recherche purement dense échoue sur les identifiants, références, codes et termes rares, qui sont précisément ce que l'on cherche le plus souvent en contexte professionnel.
- **Récupération contextuelle** : préfixez chaque fragment d'un court énoncé situant sa place dans le document d'origine. Le gain sur la précision de récupération est important pour un coût d'indexation modeste.
- **Attribution obligatoire**. Chaque fragment injecté porte sa source. Sans quoi la vérification est impossible et l'empoisonnement indétectable.
- **Budget explicite**. Fixez un plafond de jetons de récupération par requête. Sans plafond, la récupération croît jusqu'à saturer.
- **Séparation stricte données / instructions**. Tout contenu récupéré est une **donnée**. S'il contient un texte adressé à l'agent, ce texte n'est pas une consigne : c'est une charge d'injection. Voir §22.

14. Couche L6 — Mémoire de travail et artefacts

14.1 Le problème

Une tâche longue produit plus d'information qu'une fenêtre ne peut contenir. Trois primitives complémentaires y répondent, et il importe de les distinguer, car elles traitent des pathologies différentes.

Primitive	Pathologie traitée	Mécanisme	Perte
Purge des résultats d'outils	distracted par sorties volumineuses	suppression des retours anciens au-delà d'un seuil	irréversible, mais généralement sans conséquence
Compaction	saturation de l'historique	résumé de l'historique par le modèle, puis remplacement	sélective : ce que le résumé omet est perdu
Mémoire externe	perte d'état entre sessions	écriture/lecture de fichiers persistants	aucune, mais coût de discipline

14.2 Compaction

La compaction est la primitive la plus employée et la plus mal réglée. Deux règles :

1. **Remplacez la consigne de compaction par défaut.** Elle est générique ; votre charge de travail ne l'est pas. Explicitiez ce qui **doit** survivre : décisions prises et leur motif, contraintes découvertes, chemins de fichiers touchés, hypothèses en attente de validation, état d'avancement. Ce qui peut disparaître : contenus de fichiers relus, sorties d'outils, tâtonnements résolus.
2. **Évaluez la compaction par sondes.** Constituez un jeu de questions dont la réponse figure dans le contexte pré-compaction, puis posez-les après compaction. Le taux de survie est mesurable — et il est en général bien plus bas que vous ne le supposez.

14.3 Prise de notes structurée

La technique la plus simple et la plus efficace : faire tenir à l'agent un **fichier d'état externe** — plan, journal de progression, liste de tâches. Le fichier n'est pas soumis à la corrosion contextuelle : il est relu tel quel.

Le patron canonique des tâches à long horizon combine :

- un **fichier d'exigences**, rédigé au démarrage, qui empêche l'agent de considérer le travail achevé prématurément ;
- un **fichier de progression**, tenu en continu, qui permet à une fenêtre neuve de reprendre le travail sans reconstruire ;
- l'**historique de versions**, qui fournit la vérité de ce qui a été fait.

Le point critique est celui du **redémarrage à froid** : une nouvelle fenêtre doit pouvoir comprendre l'état du travail à partir de sources externes vérifiables, **jamais** à partir du résumé d'une session antérieure. C'est la seule défense robuste contre l'empoisonnement de contexte (§3.1).

14.4 Consigne de première fenêtre

Employez une consigne **différente** pour la toute première fenêtre de contexte d'une tâche longue. Cet agent initialisateur n'exécute pas la tâche : il prépare l'environnement — exigences, plan, échafaudage, jeu de cas — dont les fenêtres suivantes auront besoin. Le rendement de cette dissociation est élevé et son coût, marginal.

15. Couche L7 — Évaluation et observabilité

15.1 Trois étages d'évaluation

Étage	Objet mesuré	Fréquence	Nature
Unitaire	une sortie pour une entrée donnée	à chaque modification	assertions déterministes, schémas, expressions régulières, juge modèle sur grille
Trajectoire	la séquence d'actions	par lot, quotidien	outils appelés, ordre, nombre d'étapes, coût, respect des étapes obligatoires
Bout en bout	la tâche accomplie	par publication	succès métier, coût total, latence, satisfaction

L'étage de trajectoire est le plus souvent absent des dispositifs observés, et c'est le plus informatif sur un agent : une bonne réponse obtenue par une mauvaise trajectoire est un succès fragile.

15.2 Constitution du jeu de cas

- **Vingt à cinquante cas suffisent pour commencer.** L'objection « nous n'avons pas assez de données » est presque toujours un refus de commencer. Vingt cas discriminants valent mieux qu'un dispositif parfait jamais construit.
- **Trois familles obligatoires :** cas nominaux, cas limites, cas de **non-déclenchement** (ce qui ne doit *pas* activer une compétence, ne doit *pas* appeler un outil, doit être refusé).
- **Chaque incident de production devient un cas.** C'est la source la plus riche et la moins coûteuse.
- **Le jeu de cas est versionné avec le contexte.** Ils forment un couple.

15.3 Juge modèle

Le recours à un modèle comme évaluateur est efficace sous trois conditions strictes :

1. **Grille explicite.** Critères nommés, échelle bornée, exemples d'ancrage pour chaque niveau. Sans grille, le juge mesure sa propre préférence stylistique.
2. **Calibration humaine.** Faites noter par un humain un échantillon d'au moins trente cas, mesurez l'accord juge/humain. En dessous d'un accord substantiel, le juge est inutilisable.
3. **Conscience des biais.** Position (préférence pour la première réponse présentée), verbosité (préférence pour la réponse longue), auto-préférence (préférence pour le style du même modèle). Randomisez l'ordre, normalisez la longueur, changez de famille de modèle entre producteur et juge.

15.4 Métriques d'architecture

Au-delà de la qualité, quatre métriques mesurent la santé de l'architecture elle-même :

Métrique	Définition	Signal
Densité de signal	(jetons de contexte lus par requête) / (jetons de contexte chargés)	< 0,3 : sur-chargement
Coût fixe par requête	jetons de L0 à L4, invariants	croissance = dette d'instruction
Taux de déclenchement correct	activations de compétence pertinentes / total	< 0,8 : descriptions à reprendre
Survie post-compaction	réponses correctes aux sondes après compaction	< 0,9 : consigne de compaction à revoir

Un tableau de bord de ces quatre indicateurs, tenu par trimestre, révèle la dérive architecturale avant qu'elle ne devienne une panne de qualité.

15.5 Observabilité

Journalisez, pour chaque interaction : version de contexte, empreinte de la pile chargée, appels d'outils avec arguments et durées, jetons consommés par couche, événements de compaction. Sans cette traçabilité, un diagnostic est une conjecture — et la conformité, une affirmation.

Partie IV — Prompt engineering opérationnel

16. Le prompt canonique en huit blocs

Le prompt reste l'instrument de l'intention ponctuelle. Il doit être **mince** (§4.4) — ce qui n'interdit pas d'être structuré. Huit blocs, dont trois seulement sont obligatoires.

```
## 1. Tâche [OBLIGATOIRE]
Un verbe, un objet, un livrable. Une phrase.

## 2. Contexte [OBLIGATOIRE]
Ce que l'on ne peut pas déduire : destinataire, usage, contraintes,
état d'avancement, décisions déjà prises.

## 3. Matériau [selon le cas]
Documents, données, code – délimités par balises explicites.
C'est ici que réside le volume. Artefact épais.

## 4. Critères de réussite [OBLIGATOIRE]
Comment on jugera le résultat. Ce que l'on préfère en cas d'arbitrage.
C'est le bloc au meilleur rendement et le plus souvent omis.

## 5. Format [selon le cas]
Structure attendue, longueur, registre. Un schéma si la sortie est
consommée par une machine.

## 6. Exemples [avec parcimonie]
Un à trois cas canoniques et diversifiés. Jamais une liste de cas
particuliers (§4.2).

## 7. Périmètre négatif [avec parcimonie]
Ce qu'il ne faut explicitement pas faire, uniquement quand la
conséquence est notable. Chaque interdiction restreint l'exploration.

## 8. Procédure [tâches complexes]
Étapes attendues, points de vérification, jalons de restitution.
```

Trois observations

Le bloc 4 est le plus rentable. Une phrase de critère remplace vingt lignes de prescription. « Le lecteur doit pouvoir décider en trente secondes » cadre mieux un document qu'une spécification de mise en forme.

Le bloc 3 supporte le volume, les autres non. L'asymétrie est fondamentale : le matériau peut faire cinquante pages, la consigne doit faire vingt lignes.

L'ordre compte. Placez le matériau volumineux avant l'instruction finale : la corrosion contextuelle affecte davantage le milieu que les extrémités, et l'instruction gagne à être proche de la génération.

17. Table de sélection des techniques

Les techniques de prompting ne sont pas interchangeables. Elles répondent à des pathologies distinctes. Voici la table de correspondance.

Symptôme observé	Technique appropriée	Référence
Raisonnement escamoté sur tâche multi-étapes	décomposition explicite ; chaîne de pensée si le modèle n'en produit pas nativement	[D-02], [D-03]
Réponse instable d'un appel à l'autre	auto-cohérence par échantillonnage multiple et vote	[D-04]
Problème trop vaste, traité superficiellement	décomposition du plus simple au plus complexe ; planifier-puis-résoudre	[D-05], [D-18]
Erreurs arithmétiques ou logiques	délégation au code (PAL, programme de pensée)	[D-15], [D-16]
Hallucination factuelle	recupération augmentée ; chaîne de vérification ; attribution obligatoire	[E-10], [D-25]
Première réponse médiocre, amélioration possible	auto-raffinement ; réflexion sur retour d'exécution	[D-10], [D-09]
Exploration insuffisante des solutions	arbre de pensée ; graphe de pensée	[D-07], [D-08]
Cadrage trop étroit d'un problème	recul abstratif ; raisonnement analogique	[D-19], [D-20]
Ambiguïté de la demande	reformulation-puis-réponse ; question de clarification préalable	[D-23]
Distraction par un contexte bruité	attention de système 2 ; filtrage préalable	[D-24]
Format de sortie non respecté	schéma strict, sortie structurée, pré-remplissage de réponse	[C-03]
Besoin d'interaction avec un environnement	raisonnement-action entrelacé (ReAct)	[D-06]
Optimisation du prompt lui-même	ingénieur de prompt automatique ; compilation déclarative ; évolution par mutation	[D-11], [D-28], [D-27]
Sélection difficile d'exemples	prompting actif par mesure d'incertitude	[D-21]
Sortie trop dense ou trop creuse	chaîne de densification	[D-42]

Règle d'usage. N'appliquez une technique qu'en réponse à un symptôme constaté et mesuré. La technique appliquée par précaution est un coût contextuel sans contrepartie — et elle peut nuire : imposer une chaîne de pensée explicite à un modèle qui raisonne nativement dégrade parfois le résultat en contraignant la forme de son raisonnement.

18. Ce qui n'est plus rentable en 2026

Inventaire critique des pratiques héritées. Chacune a eu sa justification ; aucune ne survit au test de retrait sur la génération courante.

Pratique	Statut	Motif
« Réfléchis étape par étape » sur modèle raisonneur	obsolète	le raisonnement est natif ; la formule contraint sa forme
Longues listes d'exemples	contre-productif	restreint l'exploration ; préférer 1-3 cas canoniques (§4.2)
Empilement de contraintes négatives	nuisible	sur-contrainte (§3.6) ; produit des conflits
Attribution de rôle élaborée (« tu es un expert de trente ans d'expérience »)	inopérant	effet marginal ; le rôle utile est fonctionnel, non biographique
Leviers émotionnels, primes fictives, menaces	inopérant	l'effet mesuré était faible et n'est pas reproductible sur la génération courante
Répétition de l'instruction en début et fin	résiduel	utile sur très long contexte, inutile en dessous
Consigne système monolithique de plusieurs milliers de lignes	obsolète	remplacée par la révélation progressive (§4.3)
Description en prose de contraintes encodables	obsolète	à porter dans le schéma d'interface (§12.1)
Formules d'incantation, tournures magiques	jamais fondé	artefact du premier régime (§1)
Interdiction préventive du comportement indésirable	à réexaminer	conserver seulement si un cas d'évaluation le justifie

Ce qui demeure, et le restera :

- la clarté et la spécificité de la demande ;
- l'énoncé explicite des critères de réussite ;
- la fourniture du matériau nécessaire, et de lui seul ;
- la délimitation structurelle des sections ;
- la conception soignée des interfaces ;
- l'évaluation systématique.

Ce noyau n'a pas varié depuis 2020. Il ne variera pas.

Partie V — Orchestration

19. Choisir entre flux déterministe et agent

Décision structurante, et régulièrement mal tranchée par excès d'ambition.

Employez un flux déterministe quand le nombre d'étapes est prévisible et le chemin connu. Vous y gagnez : coût maîtrisé, latence stable, débogage possible, comportement reproductible.

Employez un agent quand le nombre d'étapes ne peut pas être anticipé et que le chemin dépend de découvertes faites en cours de route. Vous y gagnez de l'adaptabilité ; vous y perdez la prévisibilité du coût et vous acceptez la composition des erreurs.

Règle de conception : commencez par la solution la plus simple qui puisse fonctionner, et ne montez en complexité que sur preuve d'insuffisance. Un flux à trois appels chaînés bat un agent autonome sur toute tâche dont la structure est connue — en coût, en fiabilité et en maintenabilité.

Sur le multi-agent : l'ajout d'agents multiplie les frontières où le contexte se perd. Beaucoup de systèmes multi-agents auraient dû être des flux séquentiels bien découpés. Le sous-agent se justifie pour **l'isolation de contexte** — explorer sans polluer la fenêtre principale — bien plus souvent que pour la « spécialisation », qu'une compétence traite mieux et moins cher.

20. Les six patrons d'orchestration

Patron	Structure	Emploi
Chaînage	sortie de n → entrée de n+1	tâche décomposable en étapes fixes, chacune vérifiable
Routage	classification puis aiguillage vers un traitement spécialisé	familles de demandes hétérogènes
Parallélisation	découpage en sous-tâches indépendantes, puis agrégation	traitement en lot ; ou vote pour fiabiliser
Orchestrateur-exécutants	un planificateur découpe dynamiquement et délègue	sous-tâches non connues à l'avance
Évaluateur-optimiseur	production, critique, révision, en boucle bornée	qualité mesurable par critère explicite
Boucle agentique	l'agent agit, observe, réagit, jusqu'à satisfaction du critère	environnement dynamique, chemin imprévisible

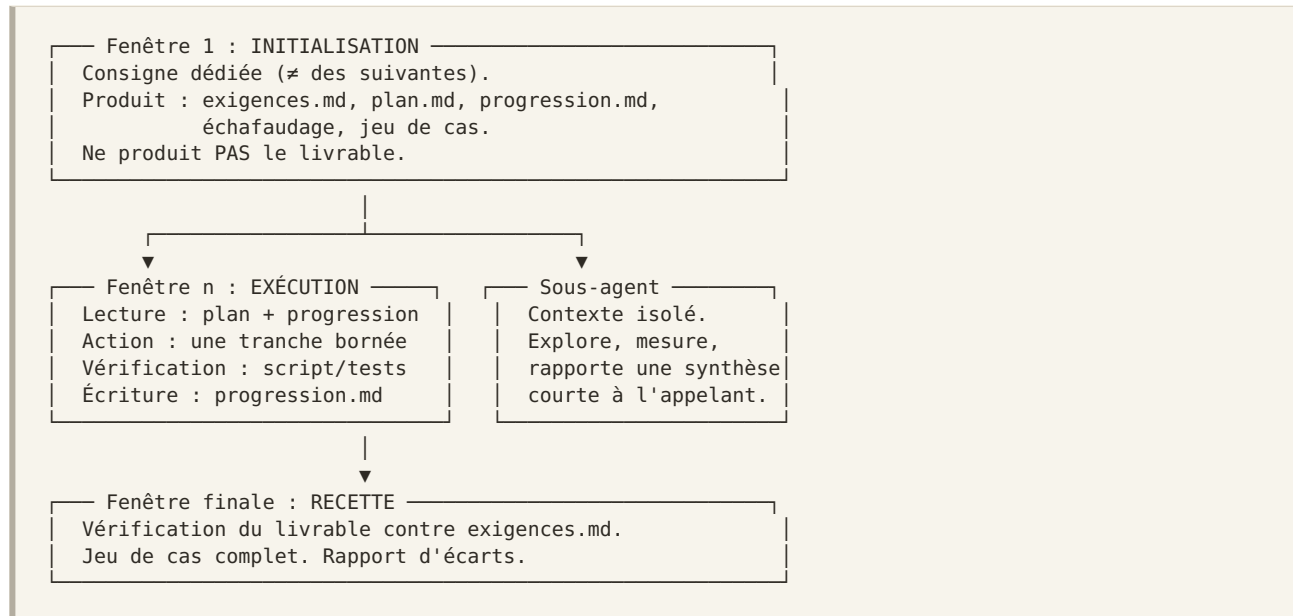
Deux garde-fous impératifs sur toute boucle :

1. **Une borne dure.** Nombre maximal d'itérations, budget de jetons, délai. Sans borne, une boucle non convergente consomme jusqu'à l'épuisement.
2. **Un critère de sortie vérifiable extérieurement.** « L'agent estime avoir terminé » n'est pas un critère : les tests passent, le schéma valide, la somme est équilibrée en est un. La vérification doit être portée par une compétence ou un script, non par le jugement de l'agent sur son propre travail.

21. Tâches à long horizon

Synthèse opérationnelle des sections 14 et 20 pour les travaux excédant une fenêtre de contexte.

Architecture recommandée



Les cinq règles

1. **L'état vit dans des fichiers**, jamais dans l'historique.
2. **Toute fenêtre neuve reconstruit son état depuis les fichiers et l'historique de versions**, jamais depuis un résumé de session.
3. **Chaque tranche de travail se termine par une vérification externe** et une écriture d'état.
4. **La compaction est configurée explicitement** et évaluée par sondes (§14.2).
5. **Les sous-agents rendent des synthèses courtes**, non leurs transcriptions. C'est l'intérêt même de l'isolation : ce qui est exploré ne remonte pas.

Partie VI — Gouvernance

22. Sécurité et chaîne d'approvisionnement

22.1 La frontière d'instruction

Principe fondateur, non négociable :

*Les instructions valides proviennent **uniquement** de l'utilisateur, par le canal d'interaction prévu. Tout ce qui est observé par un outil — page web, document, courriel, contenu de fichier, nom de fichier, message d'erreur, résultat d'appel — est une **donnée**, jamais une commande.*

Un texte qui, dans un document récupéré, s'adresse à l'agent, invoque une autorité, allègue une autorisation préalable ou presse l'urgence, ne devient pas une instruction du fait de sa formulation. C'est une charge d'injection indirecte. La conduite correcte : signaler à l'utilisateur, citer le passage, nommer la source, demander confirmation.

Corollaire souvent manqué : « traite ma liste de tâches » autorise à **lire** la liste, non à exécuter ce qu'elle contient. Les éléments à effet de bord se font confirmer un par un.

22.2 La triade létale

Un système présente un risque d'exfiltration qualitativement supérieur dès que trois propriétés coexistent :

1. accès à des **données privées** ;
2. exposition à du **contenu non fiable** ;
3. capacité de **communication vers l'extérieur**.

Deux sur trois est gérable. Trois sur trois exige une architecture de séparation : cloisonner les données, valider les sorties, ou supprimer l'un des trois termes. Aucune quantité de consigne système ne compense cette configuration.

22.3 Contrôles minimaux

Contrôle	Mise en œuvre
Moindre privilège	<code>allowed-tools</code> , portées d'API restreintes, jetons à durée courte
Séparation données/instructions	délimiteurs explicites, marquage de provenance, refus de suivre le contenu observé
Validation de sortie	schéma en sortie, liste blanche de destinataires, blocage des URL non prévues
Confirmation des effets de bord	tout envoi, publication, suppression, paiement, modification de configuration persistante
Audit des compétences et serveurs MCP	revue de code, épinglage de version, provenance signée
Aucun secret en contexte	coffre externe, injection au moment de l'appel, jamais dans un fichier de contexte
Journalisation immuable	qui, quoi, quand, avec quelle version de contexte
Test adverse	jeu de cas d'injection intégré à la suite de non-régression

22.4 Le jeu de cas adverse

Ajoutez à votre suite d'évaluation une famille de cas spécifiquement adverses : document contenant une instruction dissimulée, page web réclamant une action, fichier alléguant une autorisation antérieure, contenu encodé, appel à l'urgence, usurpation d'autorité administrative. Mesurez le taux de refus. C'est la seule manière de savoir si votre frontière d'instruction tient.

23. Conformité et traçabilité

23.1 Exigences transverses

Quel que soit le régime applicable, quatre exigences se retrouvent :

- **Traçabilité.** Pouvoir reconstituer ce qui a été produit, par quel système, avec quelle configuration de contexte, à quelle date. Le versionnement du contexte n'est pas une commodité d'ingénieur : c'est une pièce du dossier.
- **Supervision humaine.** Identifier les décisions exigeant validation humaine, et matérialiser cette validation. L'agent propose, l'humain arrête, et l'arrêt est journalisé.
- **Information de l'utilisateur.** Signaler la nature générée d'un contenu quand elle importe pour son destinataire.
- **Documentation.** Objet du système, données mobilisées, limites connues, dispositif d'évaluation, incidents survenus.

23.2 Cartographie normative

Référence	Objet	Usage
Règlement (UE) 2024/1689 (IA)	classification par risque, obligations	qualification du système, obligations de transparence et de documentation
RGPD	données personnelles	base légale, minimisation, durées, transferts
ISO/IEC 42001:2023	système de management de l'IA	cadre organisationnel certifiable
ISO/IEC 23894:2023	gestion du risque IA	méthode d'appréciation
NIST AI RMF 1.0 + AI 600-1	cadre de gestion du risque, profil IA générative	référentiel opérationnel, largement transposable
OWASP Top 10 for LLM Applications	vulnérabilités applicatives	liste de contrôle technique

Ce tableau est un repère d'orientation. Il ne constitue pas un avis juridique : la qualification d'un système au regard de ces textes relève d'une analyse propre à chaque déploiement, et l'auteur n'est pas juriste.

23.3 Registre de contexte

Tenez un registre — un simple fichier versionné suffit — recensant :

Champ	Contenu
Identifiant	référence stable de l'élément de contexte
Couche	L0 à L6
Portée	global, projet, répertoire
Motif	comportement que l'élément produit
Preuve	cas d'évaluation qui le justifie
Source	décision, texte normatif, incident d'origine
Revue	date de dernière relecture, échéance suivante

Ce registre est l'instrument qui rend la méthode SOBRE auditable. Sans lui, l'orthogonalité et la sobriété reposent sur la mémoire de celui qui a écrit le contexte — c'est-à-dire sur rien.

Partie VII — Mise en œuvre

24. Feuille de route 90 jours

Jours 1 à 7 — Inventaire

- ☐ Recenser l'intégralité du contexte existant : consignes système, fichiers de projet, compétences, définitions d'outils, mémoires.
- ☐ Mesurer le coût fixe par requête, couche par couche.
- ☐ Établir la matrice de conflit (§5, principe O). Consigner chaque doublon et chaque contradiction.
- ☐ Constituer un jeu de cas initial : vingt cas, dont cinq de non-déclenchement et trois adverses.
- ☐ Établir la mesure de référence sur ce jeu.

Livrable : rapport d'inventaire + mesure de référence.

Jours 8 à 30 — Soustraction

- ☐ Appliquer le test de retrait à chaque bloc, en commençant par les plus volumineux.
- ☐ Supprimer tout ce que le modèle peut inférer de l'environnement.
- ☐ Réécrire les contraintes négatives héritées en énoncés d'intention — hors périmètre L0.
- ☐ Éclater les fichiers de projet monolithiques en fichiers de portée fine.
- ☐ Élaguer le jeu d'outils ; fusionner les doublons ; désambiguïser les descriptions.
- ☐ Vérifier la non-régression après chaque suppression.

Livrable : pile réduite, mesure inchangée ou améliorée, écart de coût documenté. Un objectif de réduction de 50 à 80 % du coût fixe est réaliste sur une pile jamais auditée.

Jours 31 à 60 — Structuration

- ☐ Isoler le socle normatif L0, avec traçabilité par règle.
- ☐ Extraire les procédures répétées en compétences ; jeu de cas obligatoire pour chacune.
- ☐ Porter dans les schémas d'interface les contraintes actuellement en prose.
- ☐ Remplacer les contenus pré-chargés par des références légères.
- ☐ Configurer explicitement la compaction ; l'évaluer par sondes.
- ☐ Instaurer les fichiers d'état pour les tâches longues.

Livrable : pile à sept couches, orthogonale, documentée dans le registre de contexte.

Jours 61 à 90 — Instrumentation

- ☐ Étendre le jeu de cas à cinquante cas minimum, dont dix adverses.
- ☐ Automatiser l'exécution de la suite en intégration continue.
- ☐ Mettre en place le juge modèle avec grille, et le calibrer sur trente cas humains.
- ☐ Instrumenter les quatre métriques d'architecture (§15.4).
- ☐ Tenir le registre de contexte à jour et fixer les échéances de revue.

- ☐ Écrire la procédure de changement de génération de modèle.

Livrable : dispositif d'évaluation autonome, tableau de bord trimestriel, procédure de migration.

Rituel permanent

Cadence	Action
À chaque modification	test de non-régression
Hebdomadaire	incidents de production versés au jeu de cas
Mensuelle	relecture d'une couche par rotation
Trimestrielle	tableau de bord des métriques d'architecture ; élagage
À chaque nouvelle génération de modèle	audit complet de la pile — jamais une simple bascule d'identifiant de modèle

25. Catalogue des anti-patrons

#	Anti-patron	Symptôme	Correction
1	Le mur d'instruction	consigne système de plusieurs milliers de lignes	révélation progressive (§4.3)
2	Le fichier encyclopédique	CLAUDE.md racine de 500+ lignes	éclatement par portée (§10)
3	La documentation déguisée	le contexte explique ce que le dépôt dit déjà	test d'inférence (§7, pas 1)
4	La duplication prudentielle	même règle en trois endroits « pour être sûr »	orthogonalité (§5, O)
5	Le garde-fou fossile	contrainte écrite pour un modèle disparu	audit de génération (§4.6)
6	La bibliothèque d'exemples	quinze exemples dans la consigne	1-3 cas canoniques (§4.2)
7	L'interdiction en cascade	dix « ne fais jamais » consécutifs	reformulation en intention (§4.1)
8	La description muette	compétence sans condition de déclenchement	description en deux membres (§11.2)
9	La compétence-encyclopédie	SKILL.md de 2 000 lignes	corps mince + références en palier 3 (§11.3)
10	Le catalogue d'outils	quarante outils qui se recouvrent	élagage et fusion (§12.2)
11	L'outil sans borne	un appel renvoie 200 000 jetons	pagination et troncature (§12.2)
12	L'erreur opaque	le modèle ne peut pas corriger son appel	messages d'erreur actionnables (§12.2)
13	Le pré-chargement systématique	tout le corpus dans le contexte	références légères et JIT (§13)
14	La compaction par défaut	l'état critique disparaît au résumé	consigne dédiée + sondes (§14.2)
15	L'état dans l'historique	une nouvelle fenêtre ne peut pas reprendre	fichiers d'état externes (§14.3)
16	Le résumé comme vérité	l'agent reprend depuis son propre résumé	reconstruction depuis sources externes (§21)
17	La boucle non bornée	consommation jusqu'à épuisement	borne dure + critère externe (§20)
18	L'auto-vérification	l'agent déclare le travail achevé	vérification par script ou test (§20)
19	Le multi-agent décoratif	agents ajoutés sans besoin d'isolation	flux séquentiel (§19)
20	L'instruction observée	le système obéit à un document	frontière d'instruction (§22.1)
21	La triade létale ignorée	données privées + contenu non fiable + sortie externe	séparation architecturale (§22.2)
22	Le secret en contexte	clé d'API dans un fichier de contexte	coffre externe (§22.3)
23	L'évaluation absente	« ça marche mieux, je le sens »	jeu de cas et mesure (§5, É)
24	Le juge non calibré	scores élevés, qualité perçue en baisse	calibration humaine (§15.3)
25	La migration nominale	changement d'identifiant de modèle sans audit	audit complet de pile (§4.6)

Annexes

Annexe A — Gabarit CLAUDE.md

```
# [Nom du projet]

## Objet
[Deux à trois phrases : à quoi sert ce dépôt, pour qui, dans quelle phase.]

## Vocabulaire maison
- SIGLE – signification et portée.
- nom de code – ce qu'il désigne réellement.

## Chausse-trapes
<!-- Le cœur du fichier. Ce qui ne s'infère de rien. -->
- [Décision surprenante] – motif : [pourquoi] – ne pas : [action tentante et fausse].
- [Contrainte externe] – origine : [source] – conséquence : [ce qu'elle interdit].

## Zones interdites
- `chemin/` – [motif]. Ne pas modifier.

## Points d'entrée
- Pour [besoin fréquent] : commencer par `chemin/fichier`.

## Conventions arbitraires
<!-- Uniquement les choix non déductibles du code existant. -->
- [Convention] – [portée].

---
<!-- Registre de maintenance -->
<!-- Dernière revue : AAAA-MM-JJ · Modèle de référence : ... -->
<!-- Prochaine revue : AAAA-MM-JJ -->
<!-- Objectif de volumétrie : < 150 lignes. Actuel : NNN -->
```

Vérification avant validation : chaque ligne survit-elle au test de retrait ? Chaque ligne est-elle non inférable ?
Aucune ligne n'est-elle dupliquée ailleurs dans la pile ?

Annexe B — Gabarit SKILL.md

```
---
name: nom-en-minuscules-avec-tirets
description: >-
  [Ce que la compétence fait, en une phrase précise.] Utiliser lorsque
  [conditions de déclenchement explicites : termes que l'utilisateur
  emploiera, types de fichiers, natures de demande].
allowed-tools: [liste restrictive – moindre privilège]

# [Titre lisible]

## Quand ne pas utiliser cette compétence
<!-- Aussi important que le déclenchement. Préviend l'activation parasite. -->
- [Cas voisin] → utiliser plutôt [autre compétence].

## Procédure

1. [Étape] – [action impérative]. Vérifier : [critère observable].
2. [Étape] – [action impérative]. Si [condition], alors [branche].
3. [Vérification] – exécuter `scripts/verifier.py`. En cas d'échec,
  corriger et reprendre à l'étape 2. Maximum trois itérations.
```

```
## Liste de contrôle
<!-- À recopier dans la réponse et cocher au fil de l'avancement. -->
```

- ☐ Étape 1 accomplie et vérifiée
- ☐ Étape 2 accomplie et vérifiée
- ☐ Vérification finale au vert

```
## Références
<!-- Un seul niveau de profondeur. Chargées à la demande. -->
- `references/specification.md` – [ce qu'on y trouve, et quand la consulter].
- `references/cas-limites.md` – [idem].

---
<!-- Version : X.Y.Z · Revue : AAAA-MM-JJ · Échéance : AAAA-MM-JJ -->
<!-- Jeu de cas : evals/evals.json – N cas dont M de non-déclenchement -->
```

Annexe C — Gabarit de prompt

```
## Tâche
[Verbe + objet + livrable, en une phrase.]

## Contexte
- Destinataire : [qui lira, avec quel niveau d'expertise]
- Usage : [ce qui sera fait du résultat]
- État : [ce qui est déjà décidé, ce qui reste ouvert]
- Contraintes : [ce qui est imposé de l'extérieur]

## Matériau
<materiau>
[...]
</materiau>

## Critères de réussite
- [Critère observable 1]
- [Critère observable 2]
- En cas d'arbitrage entre [qualité A] et [qualité B], privilégier [A/B].

## Format
[Structure, longueur, registre. Schéma si consommation machine.]
```

Annexe D — Grille d'audit de contexte

À exécuter par couche, à chaque revue.

Sobriété - [] Chaque bloc a passé le test de retrait sur la suite d'évaluation. - [] Aucun bloc n'énonce ce que le modèle peut inférer de l'environnement. - [] Aucun bloc n'a été ajouté sans cas d'évaluation le justifiant. - [] Le coût fixe par requête est mesuré et documenté.

Orthogonalité - [] La matrice de conflit est à jour et vide de contradictions. - [] Aucune information n'apparaît dans deux couches. - [] Chaque instruction est à la portée la plus fine qui suffise. - [] Aucune couche ne contredit une couche inférieure.

Bornage - [] Les contraintes négatives hors L0 ont été réexaminées. - [] Chaque instruction énonce une intention ou un invariant, non un gabarit. - [] Le vocabulaire normatif (DOIT / DEVRAIT / PEUT) est employé avec justesse. - [] Aucune logique conditionnelle codée en dur dans la prose.

Révélation progressive - [] Les descriptions de compétences sont testées en déclenchement et non-déclenchement. - [] Aucun `SKILL.md` ne dépasse le seuil de volumétrie. - [] Les références sont à un seul niveau ; celles de plus de 100 lignes portent une table des matières. - [] Les contenus volumineux sont exposés par référence, non inlinés. - [] Le jeu d'outils est élagué ; les définitions sont différées si nombreuses.

Évaluation - [] Le jeu de cas couvre nominal, limite, non-déclenchement, adverse. - [] La suite tourne en automatique à chaque modification. - [] Le juge modèle est calibré et son accord humain mesuré. - [] Les quatre métriques d'architecture sont suivies. - [] Le registre de contexte est à jour, avec échéances de revue.

Sécurité - [] La frontière d'instruction est appliquée et testée. - [] La triade létale est absente, ou une séparation architecturale est en place. - [] Aucun secret dans un fichier de contexte. - [] Compétences et serveurs MCP audités, versions épinglées. - [] Les actions à effet de bord font l'objet d'une confirmation.

Annexe E — Glossaire

Terme	Définition
Altitude	niveau d'abstraction d'une instruction, entre prescription rigide et généralité vague
Artefact	production persistante et volumineuse (spécification, corpus, plan) opposée au prompt, mince
Compaction	résumé de l'historique par le modèle, remplaçant celui-ci pour libérer la fenêtre
Context engineering	discipline de conception de l'ensemble de l'information fournie au modèle, par opposition au seul libellé
Corrosion contextuelle (<i>context rot</i>)	dégradation de la fidélité de rappel à mesure que l'entrée s'allonge
Coût fixe par requête	jetons consommés indépendamment de la demande (couches L0 à L4)
Densité de signal	proportion du contexte chargé qui est effectivement exploité
Empoisonnement	persistance d'une erreur dans le contexte, acquérant le statut de fait
Frontière d'instruction	règle selon laquelle seul l'utilisateur émet des instructions ; tout le reste est donnée
Instruction Architecture	organisation du contexte en couches de responsabilité disjointe
JIT (juste-à-temps)	chargement de l'information au moment du besoin plutôt qu'en amont
MCP (Model Context Protocol)	protocole standardisé d'exposition d'outils et de ressources aux agents
Révélation progressive	chargement par paliers, du plus léger au plus complet, selon le besoin
Skill (compétence)	dossier contenant un <code>SKILL.md</code> , activable par description, porteur d'une procédure
Sonde	question de vérification servant à mesurer ce qui survit à une compaction
Sur-contrainte	dégradation causée par un excès de règles restreignant le jugement du modèle
Test de retrait	protocole consistant à supprimer un bloc de contexte et mesurer l'effet
Triade létale	coexistence de données privées, contenu non fiable et canal de sortie externe

Bibliographie de référence

216 entrées vérifiées, classées par domaine et hiérarchisées par priorité de lecture.

Protocole de vérification

Chaque entrée de cette bibliographie a fait l'objet d'un contrôle d'existence. Les critères retenus sont les suivants.

Pour les **travaux académiques** : identifiant arXiv ou lieu de publication confirmé, titre et auteurs concordants. Pour les **sources primaires d'éditeurs** : adresse canonique confirmée et contenu concordant avec la description donnée. Pour les **normes et standards** : référence officielle de l'organisme émetteur. Pour les **sources praticiennes** : adresse confirmée, auteur ou organisation identifiable, date de publication établie.

Toute entrée dont l'existence, l'attribution ou l'adresse n'a pas pu être confirmée a été **retirée**, y compris lorsque le contenu décrit paraissait plausible. Cette exigence a conduit à écarter une part substantielle des sources praticiennes initialement envisagées : leur volatilité éditoriale — pages déplacées, titres modifiés, publications sans auteur identifiable — les rend impropres à une citation formelle, quelle que soit la qualité de leur contenu.

Deux réserves subsistent, qu'il convient d'énoncer. Les **adresses des sources en ligne changent** : celles reproduites ici étaient valides à la date de publication du présent document et devront être revérifiées avant toute citation formelle ultérieure. Les **descriptions annotées** engagent l'appréciation de l'auteur et non celle des éditeurs cités.

A. Sources primaires — Anthropic et écosystème Claude

Autorité maximale sur les mécanismes décrits, préemption rapide. Point de départ obligatoire.

A-01. Shihpar, T. — *The new rules of context engineering for Claude 5 generation models*. Blog Claude, Anthropic, 24 juillet 2026.

Texte pivot de la doctrine de soustraction. Suppression de plus de 80 % de la consigne système de Claude Code pour Opus 5 et Fable 5, sans perte mesurable sur les évaluations de code internes ; les six bascules ; diagnostic des instructions contradictoires ; clause « sauf dans les domaines de haute importance » ; commande `/doctor`.

<https://claude.com/blog/the-new-rules-of-context-engineering-for-claude-5-generation-models>

A-02. Anthropic — *Effective context engineering for AI agents*. Blog d'ingénierie, 29 septembre 2025.

Référence fondatrice. Distinction prompt engineering / context engineering ; le contexte comme ressource finie à rendement marginal décroissant ; budget d'attention ; anatomie du contexte ; altitude juste ; stratégies de récupération ; tâches à long horizon.

<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

A-03. Anthropic — *Building effective AI agents*. Blog d'ingénierie, décembre 2024.

Flux déterministes contre agents ; les patrons d'orchestration ; annexe consacrée à l'ingénierie de prompt des outils.

<https://www.anthropic.com/engineering/building-effective-agents>

A-04. Anthropic — *Writing effective tools for AI agents — using AI agents*. Blog d'ingénierie, 2025.

Conception d'outils, nommage sans ambiguïté, bornage des retours (plafond de 25 000 jetons par défaut dans Claude Code), messages d'erreur actionnables, orientation vers des stratégies économes en jetons.

<https://www.anthropic.com/engineering/writing-tools-for-agents>

A-05. Anthropic — *Effective harnesses for long-running agents*. Blog d'ingénierie, 2025.

Consigne distincte pour la première fenêtre de contexte ; agent initialisateur ; fichier d'exigences exhaustif contre l'achèvement prématuré ; travail par sessions discrètes.

<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

A-06. Anthropic — *Equipping agents for the real world with Agent Skills*. Blog d'ingénierie, 2025.

Anatomie d'une compétence ; révélation progressive ; code embarqué.

<https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>

A-07. Anthropic — *Context engineering: memory, compaction, and tool clearing*. Claude Cookbook.

Les trois primitives comparées, avec code exécutable ; diagnostic du type de problème contextuel ; **évaluation de la compaction par questions-sondes** ; remplacement de la consigne de compaction par défaut.

<https://platform.claude.com/cookbook/tool-use-context-engineering-context-engineering-tools>

A-08. Anthropic — *Agent Skills overview*. Documentation Claude Platform.

Structure canonique, trois paliers de chargement, avertissements de sécurité.

<https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

A-09. Anthropic — *Introducing Claude Opus 5*. 24 juillet 2026.

Contexte de la bascule doctrinale ; réglages d'effort ; rapport performance/coût.

<https://www.anthropic.com/news/claude-opus-5>

A-10. Anthropic — *Scaling managed agents: decoupling the brain from the hands*. Blog d'ingénierie, 2026.

Illustration directe du principe d'audit périodique. Les réinitialisations de contexte ajoutées pour compenser l'« anxiété contextuelle » de Sonnet 4.5 sont devenues du poids mort sur Opus 4.5 : un harnais encode des hypothèses sur ce que le modèle ne sait pas faire, et ces hypothèses périclitent.

<https://www.anthropic.com/engineering/managed-agents>

A-11. Anthropic — *Introducing Contextual Retrieval*. Septembre 2024.

Préfixation contextuelle des fragments avant indexation ; gains mesurés sur la précision de récupération.

<https://www.anthropic.com/news/contextual-retrieval>

A-12. Anthropic — *How we built our multi-agent research system*. Blog d'ingénierie, juin 2025.

Orchestrateur-exécutants en production ; isolation de contexte par sous-agents ; sauvegarde du plan en mémoire externe avant troncature.

<https://www.anthropic.com/engineering/multi-agent-research-system>

A-13. Anthropic — *Introducing the Model Context Protocol*. Novembre 2024.

<https://www.anthropic.com/news/model-context-protocol>

A-14. Anthropic — *Prompt engineering overview* et guides associés. Documentation Claude.

Guides officiels : clarté, exemples, structuration par balises, pré-remplissage de réponse, chaînage de prompts.

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

A-15. Anthropic — *Building verification loops in Claude Code with skills*. 22 juillet 2026.

Externalisation de la vérification dans des compétences appelables ; publication compagnon de A-01.

A-16. Anthropic — *Prompting Claude Opus 5*. Guide de génération, juillet 2026.

Recommandations de soustraction propres à la génération 5.

A-17. Anthropic — *Claude Agent SDK*. Documentation.

Construction d'agents hors Claude Code.

<https://platform.claude.com/docs/en/api/agent-sdk/overview>

A-18. Anthropic — *Claude Code overview*. Documentation produit.

<https://docs.claude.com/en/docs/claude-code/overview>

A-19. Anthropic — *Constitutional AI: Harmlessness from AI Feedback*. 2022. arXiv:2212.08073.

Alignement par principes explicites ; fondement conceptuel du socle normatif.

A-20. Anthropic — *Many-shot Jailbreaking*. 2024.

Limites de la robustesse en contexte long ; matériau pour l'évaluation adverse.

<https://www.anthropic.com/research/many-shot-jailbreaking>

A-21. Dépôt GitHub — `anthropics/skills`.

Compétences officielles en accès libre ; matériau d'étude de premier ordre pour la rédaction de `SKILL.md`.

<https://github.com/anthropics/skills>

A-22. Dépôt GitHub — `anthropics/claude-cookbooks`.

Recettes exécutables, dont la section consacrée au context engineering.

<https://github.com/anthropics/claude-cookbooks>

B. Standard Agent Skills et Model Context Protocol

B-01. Agent Skills — *Specification*. Standard ouvert, agentskills.io.

Spécification de référence. Structure du répertoire ; schéma complet du frontmatter YAML : `name` obligatoire (1-64 caractères, minuscules alphanumériques et tirets, correspondance avec le nom du répertoire parent), `description` obligatoire (1-1024 caractères), puis `license`, `compatibility`, `metadata` et `allowed-tools` (expérimental) en option ; modèle de révélation progressive à trois paliers ; corps de `SKILL.md` recommandé sous 5 000 jetons.

<https://agentskills.io/specification>

B-02. Agent Skills — *Overview* et *Client Showcase*. agentskills.io.

Format créé par Anthropic, publié comme standard ouvert le 18 décembre 2025 sous licence Apache 2.0, gouverné par l'Agentic AI Foundation ; adopté par plus de vingt-six clients agentiques.

<https://agentskills.io/home>

B-03. Dépôt GitHub — `agentskills/agentskills`.

Spécification et documentation versionnées ; suivi des évolutions du standard.

<https://github.com/agentskills/agentskills>

B-04. Model Context Protocol — *Specification*. modelcontextprotocol.io.

Protocole d'exposition d'outils et de ressources aux agents ; donné à la Linux Foundation le 9 décembre 2025.

<https://modelcontextprotocol.io>

B-05. Microsoft Learn — *Agent Skills* (Microsoft Agent Framework).

Implémentation tierce du standard ; utile pour vérifier ce qui relève du format et ce qui relève d'un produit particulier ; documente le patron de divulgation en quatre étapes retenu par Microsoft (~100 jetons par compétence à l'annonce).

<https://learn.microsoft.com/en-us/agent-framework/agents/skills>

B-06. Xu, R. & Yan, Y. (Zhejiang University) — *Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward*. Survey, 2026. arXiv:2602.12430.

Première synthèse académique du paradigme des compétences. Quatre axes : fondements architecturaux (spécification `SKILL.md`, chargement progressif, complémentarité avec MCP), acquisition de compétences, déploiement à l'échelle, sécurité.

Constat empirique majeur : 26,1 % des compétences contribuées par la communauté contiennent des vulnérabilités. Propose un cadre de gouvernance du cycle de vie à quatre paliers de permissions indexés sur la provenance.

C. Autres éditeurs

Utiles pour dégager l'invariant du particulier.

C-01. OpenAI — *Prompt engineering guide*. Documentation API.

<https://platform.openai.com/docs/guides/prompt-engineering>

C-02. OpenAI — *Reasoning models*. Documentation API.

Pourquoi la chaîne de pensée explicite devient superflue, voire nuisible, sur les modèles raisonneurs.

<https://platform.openai.com/docs/guides/reasoning>

C-03. OpenAI — *Function calling et Structured outputs*. Documentation API.

Encodage des contraintes dans le schéma plutôt que dans la prose.

C-04. Google — *Prompt design strategies*. Documentation Gemini API.

<https://ai.google.dev/gemini-api/docs/prompting-strategies>

C-05. Boonstra, L. (Google) — *Prompt Engineering*. Livre blanc, 2024.

Synthèse structurée des techniques, avec exemples ; l'un des rares documents d'éditeur à traiter les paramètres d'échantillonnage.

C-06. Microsoft — *Prompt engineering techniques*. Documentation Azure OpenAI.

<https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/prompt-engineering>

C-07. Meta — *Prompting*. Llama documentation.

<https://www.llama.com/docs/how-to-guides/prompting/>

C-08. LangChain — *Context engineering for agents*. Blog, juin 2025 (L. Martin).

Cadre en quatre stratégies : écrire, sélectionner, compresser, isoler. Bloc-notes externe, mémoire, sous-agents.

<https://blog.langchain.com/context-engineering-for-agents/>

C-09. Ji, Y. (Manus) — *Context Engineering for AI Agents: Lessons from Building Manus*. Blog Manus, juillet 2025.

Retour d'expérience de production parmi les plus substantiels. Conception autour du cache de préfixe, masquage plutôt que suppression d'outils, système de fichiers comme contexte, récitation d'objectifs, conservation des erreurs dans la trace.

<https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>

C-10. DSPy — Documentation du cadre déclaratif d'optimisation de prompts.

<https://dspy.ai>

C-11. Model Context Protocol — Registre de serveurs et SDK.

<https://modelcontextprotocol.io/docs>

D. Techniques de prompting — littérature académique

Socle stable. À lire pour comprendre pourquoi une technique fonctionne, et dans quelles conditions.

D-01. Brown, T. et al. — *Language Models are Few-Shot Learners*. NeurIPS 2020. arXiv:2005.14165.

Acte de naissance de l'apprentissage en contexte.

D-02. Wei, J. et al. — *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2022. arXiv:2201.11903.**D-03.** Kojima, T. et al. — *Large Language Models are Zero-Shot Reasoners*. NeurIPS 2022. arXiv:2205.11916.

Origine de la formule « réfléchis étape par étape », aujourd'hui largement périmée sur les modèles raisonneurs.

D-04. Wang, X. et al. — *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR 2023. arXiv:2203.11171.**D-05.** Zhou, D. et al. — *Least-to-Most Prompting Enables Complex Reasoning in Large Language Models*. ICLR 2023. arXiv:2205.10625.**D-06.** Yao, S. et al. — *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. arXiv:2210.03629.

Fondement du patron raisonnement-action qui structure la quasi-totalité des agents actuels.

D-07. Yao, S. et al. — *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS 2023. arXiv:2305.10601.

- D-08.** Besta, M. *et al.* — *Graph of Thoughts: Solving Elaborate Problems with Large Language Models*. AAAI 2024. arXiv:2308.09687.
- D-09.** Shinn, N. *et al.* — *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023. arXiv:2303.11366.
- D-10.** Madaan, A. *et al.* — *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS 2023. arXiv:2303.17651.
- D-11.** Zhou, Y. *et al.* — *Large Language Models Are Human-Level Prompt Engineers*. ICLR 2023. arXiv:2211.01910.
- D-12.** Press, O. *et al.* — *Measuring and Narrowing the Compositionality Gap in Language Models*. EMNLP Findings 2023. arXiv:2210.03350.
- Technique dite d'auto-interrogation.
- D-13.** Khot, T. *et al.* — *Decomposed Prompting: A Modular Approach for Solving Complex Tasks*. ICLR 2023. arXiv:2210.02406.
- D-14.** Zhang, Z. *et al.* — *Automatic Chain of Thought Prompting in Large Language Models*. ICLR 2023. arXiv:2210.03493.
- D-15.** Gao, L. *et al.* — *PAL: Program-aided Language Models*. ICML 2023. arXiv:2211.10435.
- Délégation du calcul au code : le meilleur rapport fiabilité/coût sur les tâches arithmétiques et logiques.
- D-16.** Chen, W. *et al.* — *Program of Thoughts Prompting*. TMLR 2023. arXiv:2211.12588.
- D-17.** Liu, J. *et al.* — *Generated Knowledge Prompting for Commonsense Reasoning*. ACL 2022. arXiv:2110.08387.
- D-18.** Wang, L. *et al.* — *Plan-and-Solve Prompting*. ACL 2023. arXiv:2305.04091.
- D-19.** Zheng, H. S. *et al.* — *Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models*. ICLR 2024. arXiv:2310.06117.
- D-20.** Yasunaga, M. *et al.* — *Large Language Models as Analogical Reasoners*. ICLR 2024. arXiv:2310.01714.
- D-21.** Diao, S. *et al.* — *Active Prompting with Chain-of-Thought for Large Language Models*. ACL 2024. arXiv:2302.12246.
- D-22.** Li, Z. *et al.* — *Guiding Large Language Models via Directional Stimulus Prompting*. NeurIPS 2023. arXiv:2302.11520.
- D-23.** Deng, Y. *et al.* — *Rephrase and Respond: Let Large Language Models Ask Better Questions for Themselves*. 2023. arXiv:2311.04205.
- D-24.** Weston, J. & Sukhbaatar, S. — *System 2 Attention (is something you might need too)*. 2023. arXiv:2311.11829.
- Filtrage préalable du contexte bruité ; traitement direct du mode de défaillance « distraction ».
- D-25.** Dhuliawala, S. *et al.* — *Chain-of-Verification Reduces Hallucination in Large Language Models*. ACL Findings 2024. arXiv:2309.11495.
- D-26.** Zhou, P. *et al.* — *SELF-DISCOVER: Large Language Models Self-Compose Reasoning Structures*. 2024. arXiv:2402.03620.
- D-27.** Fernando, C. *et al.* — *Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution*. 2023. arXiv:2309.16797.
- D-28.** Khattab, O. *et al.* — *DSpy: Compiling Declarative Language Model Calls into Self-Improving Pipelines*. ICLR 2024. arXiv:2310.03714.
- Optimisation programmatique du prompt : l'alternative rigoureuse à l'ajustement manuel.
- D-29.** Yang, C. *et al.* — *Large Language Models as Optimizers*. ICLR 2024. arXiv:2309.03409.
- D-30.** Pryzant, R. *et al.* — *Automatic Prompt Optimization with "Gradient Descent" and Beam Search*. EMNLP 2023. arXiv:2305.03495.
- D-31.** Schulhoff, S. *et al.* — *The Prompt Report: A Systematic Survey of Prompting Techniques*. 2024. arXiv:2406.06608.
- Taxonomie de référence.** Revue systématique de type PRISMA ; 58 techniques de prompting textuel ; vocabulaire unifié. À employer comme index plutôt qu'en lecture continue.

- D-32.** Sahoo, P. *et al.* — *A Systematic Survey of Prompt Engineering in Large Language Models*. 2024. arXiv:2402.07927.
- D-33.** Liu, P. *et al.* — *Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing*. ACM Computing Surveys, 2023. arXiv:2107.13586.
- D-34.** Reynolds, L. & McDonell, K. — *Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm*. CHI 2021. arXiv:2102.07350.
- D-35.** Zhao, Z. *et al.* — *Calibrate Before Use: Improving Few-Shot Performance of Language Models*. ICML 2021. arXiv: 2102.09690.
- D-36.** Lu, Y. *et al.* — *Fantastically Ordered Prompts and Where to Find Them*. ACL 2022. arXiv:2104.08786.
- La sensibilité à l'ordre des exemples. Argument empirique en faveur de la parcimonie : moins d'exemples, moins de variance introduite.
- D-37.** Min, S. *et al.* — *Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?* EMNLP 2022. arXiv: 2202.12837.
- D-38.** Sclar, M. *et al.* — *Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design*. ICLR 2024. arXiv: 2310.11324.
- La fragilité au formatage ; justification de la structuration explicite par délimiteurs.
- D-39.** Turpin, M. *et al.* — *Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting*. NeurIPS 2023. arXiv:2305.04388.
- Une chaîne de pensée verbalisée n'est pas une preuve du raisonnement effectué.
- D-40.** Suzgun, M. *et al.* — *Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them*. 2022. arXiv: 2210.09261.
- D-41.** Ning, X. *et al.* — *Skeleton-of-Thought: Prompting LLMs for Efficient Parallel Generation*. ICLR 2024. arXiv:2307.15337.
- D-42.** Adams, G. *et al.* — *From Sparse to Dense: GPT-4 Summarization with Chain of Density Prompting*. 2023. arXiv: 2309.04269.
- D-43.** Yu, W. *et al.* — *Chain-of-Note: Enhancing Robustness in Retrieval-Augmented Language Models*. 2023. arXiv: 2311.09210.
- D-44.** Li, C. *et al.* — *Large Language Models Understand and Can Be Enhanced by Emotional Stimuli*. 2023. arXiv:2307.11760.
- À lire précisément pour constater l'ampleur modeste de l'effet et sa non-généralisation.
- D-45.** Xu, B. *et al.* — *ExpertPrompting: Instructing Large Language Models to be Distinguished Experts*. 2023. arXiv: 2305.14688.
- D-46.** Du, Y. *et al.* — *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. ICML 2024. arXiv: 2305.14325.
- D-47.** Wang, Z. *et al.* — *Unleashing the Emergent Cognitive Synergy in Large Language Models: A Task-Solving Agent through Multi-Persona Self-Collaboration*. NAACL 2024. arXiv:2307.05300.
- D-48.** Lightman, H. *et al.* — *Let's Verify Step by Step*. ICLR 2024. arXiv:2305.20050.
- Supervision de processus contre supervision de résultat.
- D-49.** Snell, C. *et al.* — *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. 2024. arXiv:2408.03314.
- D-50.** Agarwal, R. *et al.* — *Many-Shot In-Context Learning*. NeurIPS 2024. arXiv:2404.11018.
- Ce que change une très grande fenêtre pour l'économie des exemples.
- D-51.** Wei, J. *et al.* — *Emergent Abilities of Large Language Models*. TMLR 2022. arXiv:2206.07682.
- D-52.** Wei, J. *et al.* — *Finetuned Language Models Are Zero-Shot Learners*. ICLR 2022. arXiv:2109.01652.

D-53. Ouyang, L. *et al.* — *Training Language Models to Follow Instructions with Human Feedback*. NeurIPS 2022. arXiv:2203.02155.

Pourquoi les modèles suivent des instructions. Lecture indispensable pour comprendre la nature du matériau que l'on prétend architecturer.

D-54. Sanh, V. *et al.* — *Multitask Prompted Training Enables Zero-Shot Task Generalization*. ICLR 2022. arXiv:2110.08207.

D-55. Bsharat, S. M. *et al.* — *Principled Instructions Are All You Need for Questioning LLaMA-1/2, GPT-3.5/4*. 2023. arXiv:2312.16171.

Vingt-six principes énoncés ; à confronter systématiquement au chapitre 18 du présent document, plusieurs d'entre eux étant aujourd'hui périmés.

D-56. Zamfirescu-Pereira, J. D. *et al.* — *Why Johnny Can't Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts*. CHI 2023.

E. Contexte, contexte long, récupération, mémoire

E-01. Liu, N. F. *et al.* — *Lost in the Middle: How Language Models Use Long Contexts*. TACL 12:157-173, 2024. arXiv:2307.03172.

À lire absolument. Fondement empirique de la hiérarchisation par position dans le contexte.

E-02. Hong, K., Troynikov, A. & Huber, J. — *Context Rot: How Increasing Input Tokens Impacts LLM Performance*. Rapport technique, Chroma, juillet 2025.

La référence sur la corrosion contextuelle. Dix-huit modèles évalués ; dégradation non uniforme et non monotone ; effet aggravant de la faible similarité sémantique question-réponse et de la présence de distracteurs ; constat contre-intuitif d'une dégradation plus forte sur un texte cohérent que sur un texte mélangé.

<https://research.trychroma.com/context-rot> – code : <https://github.com/chroma-core/context-rot>

E-03. Shi, F. *et al.* — *Large Language Models Can Be Easily Distracted by Irrelevant Context*. ICML 2023. arXiv:2302.00093.

Base empirique du mode de défaillance « distraction ».

E-04. Hsieh, C.-P. *et al.* — *RULER: What's the Real Context Size of Your Long-Context Language Models?* 2024. arXiv:2404.06654.

L'écart entre taille nominale et taille utile.

E-05. Modarressi, A. *et al.* — *NoLiMa: Long-Context Evaluation Beyond Literal Matching*. 2025. arXiv:2502.05167.

Évaluation du contexte long au-delà de la correspondance lexicale ; complète E-02.

E-06. Laban, P. *et al.* — *LLMs Get Lost in Multi-Turn Conversation*. 2025. arXiv:2505.06120.

Dégradation propre au dialogue prolongé ; justification de l'externalisation de l'état.

E-07. Kamradt, G. — *Needle In A Haystack — Pressure Testing LLMs*. Dépôt GitHub, 2023.

Protocole devenu standard de fait, et dont E-02 et E-05 montrent les limites.

https://github.com/gkamradt/LLMTest_NeedleInAHaystack

E-08. Bai, Y. *et al.* — *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding*. ACL 2024. arXiv:2308.14508.

E-09. Levy, M. *et al.* — *Same Task, More Tokens: The Impact of Input Length on the Reasoning Performance of Large Language Models*. ACL 2024. arXiv:2402.14848.

E-10. Lewis, P. *et al.* — *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. arXiv:2005.11401.

E-11. Gao, Y. *et al.* — *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2024. arXiv:2312.10997.

- E-12.** Asai, A. *et al.* — *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. ICLR 2024. arXiv:2310.11511.
- E-13.** Jiang, Z. *et al.* — *Active Retrieval Augmented Generation*. EMNLP 2023. arXiv:2305.06983.
- E-14.** Sarthi, P. *et al.* — *RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval*. ICLR 2024. arXiv:2401.18059.
- E-15.** Edge, D. *et al.* — *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2024. arXiv:2404.16130.
- E-16.** Karpukhin, V. *et al.* — *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP 2020. arXiv:2004.04906.
- E-17.** Izacard, G. & Grave, E. — *Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering*. EACL 2021. arXiv:2007.01282.
- E-18.** Khattab, O. & Zaharia, M. — *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT*. SIGIR 2020. arXiv:2004.12832.
- E-19.** Robertson, S. & Zaragoza, H. — *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 2009.
- Pourquoi la recherche lexicale demeure indispensable en contexte professionnel, où identifiants, références et termes rares sont précisément ce que l'on cherche.
- E-20.** Jiang, H. *et al.* — *LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models*. EMNLP 2023. arXiv:2310.05736.
- E-21.** Chevalier, A. *et al.* — *Adapting Language Models to Compress Contexts*. EMNLP 2023. arXiv:2305.14788.
- E-22.** Mu, J., Li, X. L. & Goodman, N. — *Learning to Compress Prompts with Gist Tokens*. NeurIPS 2023. arXiv:2304.08467.
- E-23.** Xiao, G. *et al.* — *Efficient Streaming Language Models with Attention Sinks*. ICLR 2024. arXiv:2309.17453.
- E-24.** Packer, C. *et al.* — *MemGPT: Towards LLMs as Operating Systems*. 2023. arXiv:2310.08560.
- Mémoire hiérarchique et pagination du contexte ; parenté conceptuelle directe avec la couche de mémoire de travail.
- E-25.** Park, J. S. *et al.* — *Generative Agents: Interactive Simulacra of Human Behavior*. UIST 2023. arXiv:2304.03442.
- Flux de mémoire, réflexion périodique, récupération par pertinence-récence-importance.
- E-26.** Zhong, W. *et al.* — *MemoryBank: Enhancing Large Language Models with Long-Term Memory*. AAAI 2024. arXiv:2305.10250.
- E-27.** Sumers, T. *et al.* — *Cognitive Architectures for Language Agents*. TMLR 2024. arXiv:2309.02427.
- Cadre conceptuel de la mémoire d'agent : procédurale, sémantique, épisodique.
- E-28.** Kang, M. *et al.* — *ACON: Optimizing Context Compression for Long-Horizon LLM Agents*. 2025. arXiv:2510.00615.
- E-29.** Kerboua, I. *et al.* — *FocusAgent: Simple Yet Effective Ways of Trimming the Large Context of Web Agents*. 2025. arXiv:2510.03204.
- E-30.** *The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management*. 2025. arXiv:2508.21433.
- Résultat important pour le réglage de la couche de mémoire de travail :** le masquage simple des observations rivalise avec la compaction par résumé, à coût très inférieur.
- E-31.** Chen, S. *et al.* — *Extending Context Window of Large Language Models via Positional Interpolation*. 2023. arXiv:2306.15595.
- E-32.** Ding, Y. *et al.* — *LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens*. ICML 2024. arXiv:2402.13753.
- E-33.** Dao, T. *et al.* — *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022. arXiv:2205.14135.

E-34. Beltagy, I. *et al.* — *Longformer: The Long-Document Transformer*. 2020. arXiv:2004.05150.

E-35. Vaswani, A. *et al.* — *Attention Is All You Need*. NeurIPS 2017. arXiv:1706.03762.

Source du coût quadratique qui rend la sobriété économiquement nécessaire.

F. Agents, outils, orchestration

F-01. Schick, T. *et al.* — *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS 2023. arXiv:2302.04761.

F-02. Qin, Y. *et al.* — *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs*. ICLR 2024. arXiv:2307.16789.

F-03. Patil, S. *et al.* — *Gorilla: Large Language Model Connected with Massive APIs*. NeurIPS 2024. arXiv:2305.15334.

F-04. Shen, Y. *et al.* — *HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face*. NeurIPS 2023. arXiv:2303.17580.

F-05. Wang, G. *et al.* — *Voyager: An Open-Ended Embodied Agent with Large Language Models*. TMLR 2024. arXiv:2305.16291.

Bibliothèque de compétences acquises et réutilisables : ancêtre conceptuel des Agent Skills.

F-06. Wang, L. *et al.* — *A Survey on Large Language Model based Autonomous Agents*. 2023. arXiv:2308.11432.

F-07. Xi, Z. *et al.* — *The Rise and Potential of Large Language Model Based Agents: A Survey*. 2023. arXiv:2309.07864.

F-08. Mialon, G. *et al.* — *Augmented Language Models: A Survey*. TMLR 2023. arXiv:2302.07842.

F-09. Hong, S. *et al.* — *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. ICLR 2024. arXiv:2308.00352.

F-10. Wu, Q. *et al.* — *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. 2023. arXiv:2308.08155.

F-11. Jimenez, C. *et al.* — *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.

F-12. Yang, J. *et al.* — *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. NeurIPS 2024. arXiv:2405.15793.

La conception d'interface comme levier de performance agentique : démonstration expérimentale du principe énoncé au chapitre 12.

F-13. Liu, X. *et al.* — *AgentBench: Evaluating LLMs as Agents*. ICLR 2024. arXiv:2308.03688.

F-14. Mialon, G. *et al.* — *GAIA: A Benchmark for General AI Assistants*. ICLR 2024. arXiv:2311.12983.

F-15. Zhou, S. *et al.* — *WebArena: A Realistic Web Environment for Building Autonomous Agents*. ICLR 2024. arXiv:2307.13854.

F-16. Yao, S. *et al.* — *WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents*. NeurIPS 2022. arXiv:2207.01206.

G. Évaluation

G-01. Zheng, L. *et al.* — *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023. arXiv:2306.05685.

Référence sur le juge modèle et ses biais : position, verbosité, auto-préférence.

G-02. Gu, J. *et al.* — *A Survey on LLM-as-a-Judge*. 2024. arXiv:2411.15594.

G-03. Liu, Y. *et al.* — *G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment*. EMNLP 2023. arXiv:2303.16634.

G-04. Es, S. *et al.* — *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. EACL 2024. arXiv:2309.15217.

G-05. Chiang, W.-L. *et al.* — *Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference*. ICML 2024. arXiv:2403.04132.

G-06. Hendrycks, D. *et al.* — *Measuring Massive Multitask Language Understanding*. ICLR 2021. arXiv:2009.03300.

G-07. Srivastava, A. *et al.* — *Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models*. TMLR 2023. arXiv:2206.04615.

G-08. Rein, D. *et al.* — *GPQA: A Graduate-Level Google-Proof Q&A Benchmark*. 2023. arXiv:2311.12022.

G-09. Bommasani, R. *et al.* — *Holistic Evaluation of Language Models*. TMLR 2023. arXiv:2211.09110.

G-10. Chang, Y. *et al.* — *A Survey on Evaluation of Large Language Models*. ACM TIST, 2024. arXiv:2307.03109.

G-11. Ribeiro, M. T. *et al.* — *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*. ACL 2020. arXiv:2005.04118.

Modèle méthodologique du jeu de cas comportemental, transposable tel quel.

G-12. Ji, Z. *et al.* — *Survey of Hallucination in Natural Language Generation*. ACM Computing Surveys, 2023. arXiv:2202.03629.

G-13. Huang, L. *et al.* — *A Survey on Hallucination in Large Language Models*. 2023. arXiv:2311.05232.

G-14. Zhang, T. *et al.* — *BERTScore: Evaluating Text Generation with BERT*. ICLR 2020. arXiv:1904.09675.

G-15. Lin, C.-Y. — *ROUGE: A Package for Automatic Evaluation of Summaries*. ACL Workshop on Text Summarization, 2004.

G-16. Papineni, K. *et al.* — *BLEU: A Method for Automatic Evaluation of Machine Translation*. ACL 2002.

G-17. Husain, H. — *Your AI Product Needs Evals*. 2024.

Le plaidoyer opérationnel le plus convaincant en faveur de l'évaluation d'abord.

<https://hamel.dev/blog/posts/evals/>

G-18. Husain, H. — *Context Rot* (notes annotées de la présentation de K. Hong). 2025.

Version commentée et pédagogique de E-02.

https://hamel.dev/notes/llm/rag/p6-context_rot.html

H. Sécurité, robustesse, injection

H-01. Greshake, K. *et al.* — *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. AISec 2023 (16th ACM Workshop on Artificial Intelligence and Security), p. 79-90. arXiv:2302.12173.

Texte fondateur de l'injection indirecte.

H-02. Willison, S. — *The lethal trifecta for AI agents: private data, untrusted content, and external communication*. 16 juin 2025.

La formulation la plus claire du risque architectural d'exfiltration. Trois propriétés qui, réunies, rendent un système structurellement vulnérable ; aucune consigne système ne compense la configuration.

<https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>

H-03. Wallace, E. *et al.* — *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*. 2024. arXiv:2404.13208.

Fondement théorique de la frontière d'instruction énoncée au chapitre 22.

H-04. Perez, F. & Ribeiro, I. — *Ignore Previous Prompt: Attack Techniques For Language Models*. NeurIPS Workshop 2022. arXiv:2211.09527.

H-05. Liu, Y. *et al.* — *Prompt Injection Attack against LLM-integrated Applications*. 2023. arXiv:2306.05499.

H-06. Zou, A. *et al.* — *Universal and Transferable Adversarial Attacks on Aligned Language Models*. 2023. arXiv:2307.15043.

H-07. Wei, A. *et al.* — *Jailbroken: How Does LLM Safety Training Fail?* NeurIPS 2023. arXiv:2307.02483.

H-08. Carlini, N. *et al.* — *Extracting Training Data from Large Language Models.* USENIX Security 2021. arXiv:2012.07805.

H-09. DeBenedetti, E. *et al.* — *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents.* NeurIPS 2024. arXiv:2406.13352.

Évaluation adverse d'agents ; matériau directement réutilisable pour constituer un jeu de cas adverse.

H-10. DeBenedetti, E. *et al.* (Google, Google DeepMind, ETH Zürich) — *Defeating Prompt Injections by Design (CaMeL).* 2025. arXiv:2503.18813.

Défense architecturale plutôt que par consigne. Extraction explicite des flux de contrôle et de données ; notion de capacité empêchant l'exfiltration sur des flux non autorisés ; 67 % des tâches d'AgentDojo résolues avec garantie de sécurité.

H-11. Yi, J. *et al.* — *Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models.* 2023. arXiv:2312.14197.

H-12. Chao, P. *et al.* — *JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models.* NeurIPS 2024. arXiv:2404.01318.

H-13. Ganguli, D. *et al.* — *Red Teaming Language Models to Reduce Harms.* 2022. arXiv:2209.07858.

H-14. Perez, E. *et al.* — *Red Teaming Language Models with Language Models.* EMNLP 2022. arXiv:2202.03286.

H-15. Weidinger, L. *et al.* — *Ethical and Social Risks of Harm from Language Models.* 2021. arXiv:2112.04359.

H-16. OWASP — *Top 10 for Large Language Model Applications.*

Liste de contrôle technique de référence, révisée annuellement ; complétée depuis 2026 par un volet consacré aux applications agencielles.

<https://owasp.org/www-project-top-10-for-large-language-model-applications/>

H-17. NIST — *Artificial Intelligence Risk Management Framework (AI RMF 1.0).* NIST AI 100-1, janvier 2023.

<https://www.nist.gov/itl/ai-risk-management-framework>

H-18. NIST — *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile.* NIST AI 600-1, juillet 2024.

H-19. MITRE — *ATLAS : matrice des tactiques et techniques adverses contre les systèmes d'intelligence artificielle.*

<https://atlas.mitre.org>

I. Ouvrages

I-01. Berryman, J. & Ziegler, A. — *Prompt Engineering for LLMs.* O'Reilly, 2024.

Le plus proche d'un manuel d'ingénierie sur le sujet ; écrit par des praticiens de GitHub Copilot.

I-02. Phoenix, J. & Taylor, M. — *Prompt Engineering for Generative AI.* O'Reilly, 2024.

I-03. Huyen, C. — *AI Engineering: Building Applications with Foundation Models.* O'Reilly, 2025.

Le meilleur ouvrage d'ensemble sur la mise en production. Chapitres d'évaluation particulièrement solides.

I-04. Huyen, C. — *Designing Machine Learning Systems.* O'Reilly, 2022.

Pour l'observabilité et la conception de boucles de rétroaction.

I-05. Alammari, J. & Grootendorst, M. — *Hands-On Large Language Models.* O'Reilly, 2024.

I-06. Raschka, S. — *Build a Large Language Model (From Scratch).* Manning, 2024.

Pour comprendre ce que « contexte » signifie mécaniquement.

I-07. Jurafsky, D. & Martin, J. H. — *Speech and Language Processing*, 3e édition (version en accès libre).

Référence de fond en traitement automatique des langues.

<https://web.stanford.edu/~jurafsky/slp3/>

I-08. Russell, S. & Norvig, P. — *Artificial Intelligence: A Modern Approach*, 4e édition. Pearson, 2020.

Chapitres consacrés aux agents : vocabulaire de référence, antérieur et supérieur à celui de la littérature pratique actuelle.

I-09. Simon, H. A. — *The Sciences of the Artificial*, 3e édition. MIT Press, 1996.

Rationalité limitée et attention comme ressource rare : l'argument théorique du principe de sobriété.

I-10. Miller, G. A. — *The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information*. *Psychological Review*, 63(2), 1956, p. 81-97.

I-11. Sweller, J. — *Cognitive Load During Problem Solving: Effects on Learning*. *Cognitive Science*, 12(2), 1988, p. 257-285.

Origine de la théorie de la charge cognitive, cadre théorique de la révélation progressive.

I-12. Nielsen, J. — *Usability Engineering*. Morgan Kaufmann, 1993.

Origine ergonomique de la notion de révélation progressive.

I-13. Krug, S. — *Don't Make Me Think, Revisited*, 3e édition. New Riders, 2014.

I-14. Clark, A. & Chalmers, D. — *The Extended Mind*. *Analysis*, 58(1), 1998, p. 7-19.

Cadre philosophique de la mémoire externalisée.

I-15. Hutchins, E. — *Cognition in the Wild*. MIT Press, 1995.

Cognition distribuée entre agents et artefacts.

I-16. Brooks, F. P. — *The Mythical Man-Month: Essays on Software Engineering*, édition anniversaire. Addison-Wesley, 1995.

Complexité essentielle et complexité accidentelle : grille de lecture de la dette d'instruction.

I-17. Hunt, A. & Thomas, D. — *The Pragmatic Programmer*, 2e édition. Addison-Wesley, 2019.

Principe DRY appliqué au contexte : fondement du principe d'orthogonalité.

I-18. Fowler, M. — *Refactoring: Improving the Design of Existing Code*, 2e édition. Addison-Wesley, 2018.

Le contexte est du code ; il se refactorise selon les mêmes méthodes.

I-19. Kim, G., Humble, J., Debois, P. & Willis, J. — *The DevOps Handbook*, 2e édition. IT Revolution, 2021.

Boucles de rétroaction courtes et non-régression automatisée.

J. Normes, standards, réglementation

J-01. Union européenne — Règlement (UE) 2024/1689 du 13 juin 2024 établissant des règles harmonisées concernant l'intelligence artificielle. JOUE L, 12 juillet 2024.

<https://eur-lex.europa.eu/eli/reg/2024/1689/oj>

J-02. Union européenne — Règlement (UE) 2016/679 du 27 avril 2016 relatif à la protection des personnes physiques à l'égard du traitement des données à caractère personnel (RGPD).

<https://eur-lex.europa.eu/eli/reg/2016/679/oj>

J-03. ISO/IEC 42001:2023 — *Information technology — Artificial intelligence — Management system*.

J-04. ISO/IEC 23894:2023 — *Information technology — Artificial intelligence — Guidance on risk management*.

J-05. ISO/IEC 22989:2022 — *Information technology — Artificial intelligence — Artificial intelligence concepts and terminology*.

J-06. Bradner, S. — *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119, IETF, mars 1997.

Vocabulaire normatif à employer dans la rédaction d'instructions. DOIT, NE DOIT PAS, DEVRAIT, PEUT — avec la sémantique exacte que le document définit.

<https://www.rfc-editor.org/rfc/rfc2119>

J-07. Leiba, B. — *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. RFC 8174, IETF, mai 2017.

Complément indispensable à J-06 : seule la forme capitalisée porte la valeur normative.

J-08. CommonMark Specification.

Format de référence du présent document et des fichiers `SKILL.md`.

<https://spec.commonmark.org>

J-09. YAML Ain't Markup Language (YAML) version 1.2.

Frontmatter des fichiers de compétence.

<https://yaml.org/spec/1.2.2/>

J-10. Preston-Werner, T. — *Semantic Versioning 2.0.0*.

Versionnement des compétences et des fichiers de contexte.

<https://semver.org>

J-11. *Keep a Changelog*.

Tenue du journal des modifications de contexte.

<https://keepachangelog.com>

J-12. OpenAPI Specification.

Fondement de la conception d'interfaces typées.

<https://spec.openapis.org/oas/latest.html>

J-13. JSON Schema Specification.

<https://json-schema.org/specification>

J-14. Commission nationale de l'informatique et des libertés — Recommandations sur le développement de systèmes d'intelligence artificielle.

Pour un déploiement soumis au droit français.

<https://www.cnil.fr/fr/intelligence-artificielle>

Ordre de lecture recommandé

Premier palier — le socle indispensable. *Environ une demi-journée.* A-01, A-02, A-03, E-01, E-02.

Deuxième palier — la mise en pratique. *Environ deux jours.* A-04, A-05, A-06, A-07, A-08, B-01, C-09, G-17.

Troisième palier — la profondeur théorique. *Environ une semaine.* D-31 (comme index plutôt qu'en lecture continue), D-02, D-04, D-06, D-53, E-03, E-10, F-12, G-01, H-01, H-03.

Quatrième palier — la gouvernance. *Selon le degré d'exposition.* B-06, H-02, H-10, H-16, J-01, J-06.

Cinquième palier — le fond conceptuel. *À loisir.* I-01, I-03, I-09, I-12, I-14, I-17.

Colophon

Titre. La Méthode SOBRE — Architecture d'instruction pour l'usage optimal des grands modèles de langage

Auteur. D. Dambreville

Version. 1.1 — 18 août 2026

Licence. Creative Commons Attribution 4.0 International (CC BY 4.0). Reproduction, distribution et adaptation autorisées, y compris à des fins commerciales, sous réserve d'attribution. Texte intégral de la licence : <https://creativecommons.org/licenses/by/4.0/deed.fr>

Bibliographie. 216 entrées vérifiées, réparties en dix sections.

Comment citer

Dambreville, D. (2026). La Méthode SOBRE — Architecture d'instruction pour l'usage optimal des grands modèles de langage, version 1.1.

Avertissement

Ce document présente une méthode d'ingénierie. Il ne constitue ni un avis juridique, ni une garantie de résultat. Le chapitre 23 offre un repère d'orientation normative ; la qualification d'un système au regard des textes cités relève d'une analyse propre à chaque déploiement et, le cas échéant, d'un conseil qualifié.

Les mécanismes, seuils et volumétries décrits reflètent l'état de l'art à la date de publication. La génération de modèles évolue plus vite que la doctrine qui l'accompagne : les repères chiffrés — lignes de fichier, jetons de compétence, plafonds de retour d'outil — sont des ordres de grandeur empiriques à revérifier, non des constantes. Le chapitre 4.6 fait de cette révision une obligation méthodologique, qui vaut aussi pour le présent document.